VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



ASSIGNMENT I

DEEP LEARNING AND ITS APPLICATIONS — CO3133

FOUNDATIONS OF DEEP LEARNING PIPELINES AND ARCHITECTURES

From Linear Models to Modern Sequence Models: A Comparative Study for Image Classification

---

MAJOR: COMPUTER SCIENCE

CLASS: CC04

SUPERVISOR: PhD. Lê Thành Sách

— o0o —

STUDENT: HOÀNG XUÂN BÁCH 2352082
NGUYỄN VIỆT HÙNG 2352424
LÊ NGUYỄN GIA PHÚC 2352931

HO CHI MINH CITY, September 2026

Contents

1 Problem and Data Description **1**

1.1 Problem formulation . . . 1

1.1.1 Real-world motivation . . . 1

1.1.2 Input . . . 1

1.1.3 Output . . . 1

1.1.4 Prediction unit . . . 2

1.1.5 Mathematical formulation . . . 2

1.1.6 Main difficulties . . . 2

1.2 Dataset description . . . 3

1.2.1 Datasets used in this assignment . . . 3

1.2.2 Source, version, and license . . . 3

1.2.3 Collection and labeling pipeline . . . 3

1.2.4 Sample counts and annotation format . . . 4

1.2.5 Storage size and memory footprint . . . 4

1.2.6 Dataset bias and limitations . . . 4

1.3 Exploratory Data Analysis . . . 5

1.3.1 Input size and data integrity . . . 5

1.3.2 Class distribution . . . 5

1.3.3 Imbalance analysis . . . 6

1.3.4 Pixel-intensity distribution (training split only) . . . 7

1.3.5 Representative samples . . . 8

1.3.6 Summary . . . 9

1.4 Data splitting . . . 10

1.4.1 Dataset Partitioning and Split Strategy . . . 10

1.4.2 Seed and Deterministic Reproducibility . . . 10

1.4.3 Stratification . . . 10

1.4.4 Split Unit . . . 11

1.4.5 Data Leakage Prevention . . . 11

1.4.6 Subset Selection Rules . . . 11

1.5 Data Preprocessing . . . 12

1.5.1 Data Cleaning and Invalid Annotation Handling . . . 12

1.5.2 Input Normalization . . . 12

1.5.3 Data Augmentation . . . 12

1.5.4 Input Tensor Shapes . . . 13
1.5.5 Example Batch after Preprocessing . . . 13

2 Methodology **14**

2.1 Pipeline Overview . . . 14
2.2 Baseline (Linear / softmax classifier) . . . 14
2.2.1 Architecture and input representation . . . 14
2.2.2 Parameter count . . . 15
2.2.3 Strengths and limitations . . . 15
2.2.3.a Strengths. . . . 15
2.2.3.b Limitations. . . . 15
2.2.4 Self-implemented components and library dependencies . . . 16
2.3 Multilayer Perceptron (MLP) . . . 16
2.3.1 Architecture and input representation. . . . 16
2.3.2 Parameter Budget Analysis. . . . 17
2.3.3 Strengths and limitations . . . 17
2.3.3.a Strengths . . . 17
2.3.3.b Limitations . . . 18
2.3.4 Self-implemented components and library dependencies . . . 18
2.4 Convolutional Neural Network (CNN) . . . 18
2.4.1 Architecture and input representation . . . 18
2.4.2 Parameter Budget Analysis. . . . 20
2.4.3 Strengths and limitations . . . 20
2.4.3.a Strengths . . . 20
2.4.3.b Limitations . . . 21
2.4.4 Self-implemented components and library dependencies . . . 21
2.5 Long Short-Term Memory (LSTM) . . . 22
2.5.1 Architecture Description and input representation . . . 22
2.5.2 Parameter Budget Analysis. . . . 24
2.5.3 Strengths and limitations . . . 25
2.5.3.a Strengths . . . 25
2.5.3.b Limitations . . . 25
2.5.4 Self-implemented components and library dependencies . . . 26
2.6 Transformer . . . 26
2.6.1 Architecture Description and Input representation . . . 26

2.6.2 Parameter Budget Analysis. 29
2.6.3 Strengths and limitations 29
2.6.3.a Strengths 29
2.6.3.b Limitations 30
2.6.4 Self-implemented components and library dependencies 30
2.7 Shared loss and evaluation metrics 31
2.7.1 Loss function 31
2.7.2 Predictive evaluation metrics 31
2.8 Training setup 33
2.9 Experimental Design 35

3 Implementation Results **37**
3.1 Training behavior 37
3.1.1 Baseline (Linear / softmax classifier) 37
3.1.2 Multilayer Perceptron 39
3.1.3 Convolutional Neural Network (CNN) 41
3.1.4 Long Short Term Memory (LSTM) 43
3.1.5 Transformer 45
3.2 Quantitative results 46
3.3 Qualitative results 54
3.3.1 Baseline (Linear / softmax classifier) 55
3.3.2 Multilayer Perceptron (MLP) 56
3.3.3 Convolutional Neural Network (CNN) 58
3.3.4 Long Short Term Memory (LSTM) 59
3.3.5 Transformer 61
3.4 Comparison and discussion 62
3.4.1 Overall predictive performance 62
3.4.2 Class-specific improvements and remaining errors 63
3.4.3 Architecture and input representation 63
3.4.4 Loss, training behavior, and prediction confidence 64
3.4.5 Accuracy, execution time, and parameter trade-offs 65
3.4.6 Hypothesis assessment and limitations 65
3.5 Error Analysis 66
3.5.1 Confusion between visually similar classes 66
3.5.2 Incorrect predictions with high softmax scores 67

4 Ablation study **68**

5 Limitations and conclusion **69**

5.1 Limitations 69

5.1.1 Data limitations. 69

5.1.2 Method limitations. 69

5.2 Generalization and future work 69

5.3 Conclusion 70

6 AI Usage Disclosure **71**

6.1 Review and Refinement of the Training and Evaluation Pipeline 71

6.2 Student-Proposed Architectures and Technical Verification 72

6.3 Feedback on Results Interpretation and Discussion 72

6.4 English Language Editing 73

6.5 AI-Assisted Github Pages Skeleton 74

6.6 Consistency Verification Between Experimental Outputs and Report 74

6.7 Consistency Verification Between Experimental Outputs and GitHub Pages 75

6.8 LaTeX Formatting Assistance 76

List of Figures

1 Number of images per class in the official training (60,000) and test (10,000) sets. 5
2 Images per class in the training split (48,000) and the validation split (12,000). 6
3 Pixel-intensity histogram (log scale) of the 48,000 training images. The tall bar at 0 is the background (about 50.2% of pixels). 7
4 Random training images, classes 0–4 (T-shirt/top, Trouser, Pullover, Dress, Coat). 8
5 Random training images, classes 5–9 (Sandal, Shirt, Sneaker, Bag, Ankle boot). 9
6 Pipeline Overview 14
7 Learning curves of the linear classifier for training seeds 42, 43, and 44. 37
8 Learning curves of the MLP (Dashed vertical lines indicate the epochs selected by minimum validation loss) 39
9 Learning curves of the CNN (Dashed vertical lines indicate the epochs selected by minimum validation loss) 41
10 Learning curves of the LSTM (Dashed vertical lines indicate the epochs selected by minimum validation loss) 43
11 Learning curves of the Transformer (Dashed vertical lines indicate the epochs selected by minimum validation loss) 45
12 Test confusion matrix across three training seeds of the linear classifier. 51
13 Test confusion matrix across three training seeds of the MLP. 52
14 Test confusion matrix across three training seeds of the CNN. 52
15 Test confusion matrix across three training seeds of the LSTM. 53
16 Test confusion matrix across three training seeds of the Transformer. 53
17 Correct predictions of the linear classifier for training seed 42. 55
18 Difficult but correctly classified candidates of the linear classifier for training seed 42 . . 55
19 Incorrect predictions of the linear classifier for training seed 42 56
20 Correct predictions of the MLP for training seed 42. 56
21 Difficult but correctly classified candidates of the MLP for training seed 42 57
22 Incorrect predictions of the MLP for training seed 42 57
23 Correct predictions of the CNN for training seed 42. 58
24 Difficult but correctly classified candidates of the CNN for training seed 42 58
25 Incorrect predictions of the CNN for training seed 42 59
26 Correct predictions of the LSTM for training seed 42. 59
27 Difficult but correctly classified candidates of the LSTM for training seed 42 60
28 Incorrect predictions of the LSTM for training seed 42 60

29 Correct predictions of the Transformer for training seed 42. 61
30 Difficult but correctly classified candidates of the Transformer for training seed 42 61
31 Incorrect predictions of the Transformer for training seed 42 62

List of Tables

1 Fashion-MNIST class mapping and verified sample counts. 4
2 Class balance of each partition. 6
3 Mean pixel intensity per class on the 48,000 training images. 8
4 Dataset Partitioning and Class Distribution Summary 10
5 Layer-by-layer Trainable Parameter Breakdown of the `MLP` 17
6 Layer-by-layer Trainable Parameter Breakdown of `SimpleCNN` 20
7 Layer-by-layer Trainable Parameter Breakdown of `RowLSTM` 25
8 Layer-by-layer Trainable Parameter Breakdown of `PatchTransformer` 29
9 Standardized Training and Optimization Hyperparameters 34
10 Hypotheses and experimental settings for the main model comparison and dropout ablation. 35
11 Evaluation metrics and conclusion criteria for the main model comparison and dropout ablation. 36
12 Predictive performance, model size, and computational cost of the five models, reported as mean ± sample standard deviation across training seeds 42, 43, and 44. 47
13 Per-class test metrics of the linear classifier, reported as mean ± sample standard deviation across three training seeds. 48
14 Per-class test metrics of the MLP, reported as mean ± sample standard deviation across three training seeds. 49
15 Per-class test metrics of the CNN, reported as mean ± sample standard deviation across three training seeds. 49
16 Per-class test metrics of the LSTM, reported as mean ± sample standard deviation across three training seeds. 50
17 Per-class test metrics of the Transformer, reported as mean ± sample standard deviation across three training seeds. 50
18 Test predictive performance of the NoDropout variants, reported as mean ± sample standard deviation across training seeds 42, 43, and 44 68
19 Paired predictive-performance differences between NoDropout and Baseline, reported as mean ± sample standard deviation across training seeds 42, 43, and 44. 68

1. Problem and Data Description

1.1. Problem formulation

1.1.1. Real-world motivation

Clothing-image recognition is a compact but meaningful example of visual perception. A reliable classifier must distinguish visually similar clothing categories—for example, a Shirt from a T-shirt/top or a Pullover from a Coat—from low-resolution grayscale images.

The practical goal of this problem is to develop a model that can automatically recognize the clothing category depicted in a given Fashion-MNIST image. Beyond obtaining high predictive performance, the assignment aims to understand how different neural-network architectures process the same visual information and how their architectural assumptions affect predictive quality and computational efficiency.

The task uses the Fashion-MNIST dataset, which contains grayscale images from ten clothing categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot.

1.1.2. Input

The input is a single Fashion-MNIST clothing image. Each image has a spatial resolution of 28×28 pixels and contains a single grayscale channel.

After preprocessing, the image can be represented as a tensor

$$\mathbf{x} \in \mathbb{R}^{1\times 28\times 28}. \tag{1}$$

Depending on the model architecture, this image may be transformed into different representations. Linear and MLP models use a flattened vector $\mathbf{x} \in \mathbb{R}^{784}$, whereas CNN models preserve the original two-dimensional spatial structure. LSTM/GRU or Transformer models may represent the image as a sequence of rows, columns, or image patches.

The initial input remains the same across all models: only the representation is adapted to the requirements of each architecture.

1.1.3. Output

The model produces ten real-valued logits, with one logit corresponding to each of the ten Fashion-MNIST clothing categories:

$$\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^{10}. \tag{2}$$

The predicted clothing category is the class associated with the largest logit:

$$\widehat{y} = \underset{c \in \{0,\ldots,9\}}{\arg\max}\, z_c. \tag{3}$$

The corresponding target is a single integer class label

$$y \in \{0,1,\ldots,9\}. \tag{4}$$

1.1.4. Prediction unit

The prediction unit is one individual Fashion-MNIST image. For each input image, the model predicts exactly one of the ten clothing categories. Thus, the problem is a single-label multiclass classification problem, where each image has one ground-truth class and the model must assign one predicted class to that image.

1.1.5. Mathematical formulation

Let the labeled dataset be

$$\mathcal{D} = \{(\mathbf{x}_i, y_i) \mid i = 1,\ldots,N\}, \qquad \mathbf{x}_i \in \mathcal{X} \subseteq \mathbb{R}^{1\times 28\times 28}, \qquad y_i \in \mathcal{Y} = \{0,1,\ldots,9\}. \tag{5}$$

A model parameterized by θ maps an input image to ten class logits:

$$f_\theta : \mathcal{X} \to \mathbb{R}^{10}, \qquad \mathbf{z}_i = f_\theta(\mathbf{x}_i). \tag{6}$$

The logits are converted into class probabilities using the softmax function:

$$p_{i,c} = \frac{\exp(z_{i,c})}{\sum_{k=0}^{9} \exp(z_{i,k})}, \qquad c \in \mathcal{Y}. \tag{7}$$

The predicted class is therefore

$$\widehat{y}_i = \underset{c\in\mathcal{Y}}{\arg\max}\, p_{i,c} = \underset{c\in\mathcal{Y}}{\arg\max}\, z_{i,c}. \tag{8}$$

1.1.6. Main difficulties

The main difficulties of this problem are:

- **Low resolution and visual ambiguity:** Fashion-MNIST images contain limited visual detail, while several clothing categories have similar shapes and silhouettes. This can lead to systematic confusions, such as Shirt and T-shirt/top or Pullover and Coat.
- **Representation mismatch:** Different architectures require different input representations. Flattening an image removes its explicit two-dimensional neighborhood structure, while converting an image into a row or column sequence imposes an artificial ordering. These representation choices may advantage or disadvantage particular architectures.

1.2. Dataset description

1.2.1. Datasets used in this assignment

Following the course requirements, two datasets from the MNIST family are considered. MNIST (handwritten digits, 10 classes) is strictly reserved for initial pipeline development, unit sanity checks, and debugging (DataLoader behavior, forward-backward step integrity, and evaluation metrics) prior to running larger compute jobs. Fashion-MNIST serves as the primary dataset for all comparative experimental results presented in this report. All five mandatory model architectures—Linear/Softmax classifier, Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN), Long Short Term Memory (LSTM), and Vision Transformer (ViT) —are trained, validated, and benchmarked on Fashion-MNIST under a strictly identical data split and evaluation protocol.

1.2.2. Source, version, and license

Fashion-MNIST was introduced in 2017 by Zalando Research (Han Xiao, Kashif Rasul, and Roland Vollgraf) as a direct drop-in replacement for the original MNIST digit dataset. It was designed to provide a more challenging classification benchmark while retaining the exact 28×28 grayscale image dimensionality, 10-class labels, and 60,000/10,000 train/test split. The dataset is distributed under the MIT License (Copyright © 2017 Zalando SE) and is hosted publicly at https://github.com/zalandoresearch/fashion-mnist. In our experimental pipeline, we pull the official release directly via `torchvision.datasets.FashionMNIST`

1.2.3. Collection and labeling pipeline

The dataset comprises single-channel grayscale e-commerce product photos sourced from Zalando's online retail catalog. The original high-resolution, full-color studio product images underwent automated background removal, aspect-ratio-preserving resizing, centering, and downsampling to 28×28 pixels. Each sample is associated with exactly one ground-truth label among 10 mutually exclusive garment categories directly mapped from Zalando's product categorization hierarchy. No secondary manual re-annotation or label curation was performed by our team.

1.2.4. Sample counts and annotation format

Fashion-MNIST contains a total of 70,000 samples, divided by default into an official training partition of 60,000 images and an official test partition of 10,000 images. Each image is paired with a scalar target $y \in \{0, 1, \ldots, 9\}$. The class mappings and verified counts are summarized in Table 1.

Table 1: Fashion-MNIST class mapping and verified sample counts.

Label	Class Name	Official Train	Official Test
0	T-shirt/top	6,000	1,000
1	Trouser	6,000	1,000
2	Pullover	6,000	1,000
3	Dress	6,000	1,000
4	Coat	6,000	1,000
5	Sandal	6,000	1,000
6	Shirt	6,000	1,000
7	Sneaker	6,000	1,000
8	Bag	6,000	1,000
9	Ankle boot	6,000	1,000
Total		**60,000**	**10,000**

1.2.5. Storage size and memory footprint

When downloaded and extracted by `torchvision`, the raw directory on disk occupies approximately 85.8 megabytes; in memory, each sample is initially represented as a 28×28 `uint8` array with pixel intensities ranging in $[0, 255]$. Upon ingestion into the PyTorch pipeline via `ToTensor`, images are cast to tensors with dimensions $1 \times 28 \times 28$ and values scaled to $[0.0, 1.0]$. The memory footprint of the full 60,000 training images in float32 format is approximately $60{,}000 \times 28 \times 28 \times 4$ bytes ≈ 188.16 MB.

1.2.6. Dataset bias and limitations

Several structural constraints of Fashion-MNIST must be accounted for during modeling:

- **Commercial catalog bias:** Images originate exclusively from studio photography with uniform lighting, centered bounding boxes, and isolated frontal/side perspectives. Consequently, models trained on this benchmark do not automatically generalize to consumer photos characterized by arbitrary angles, cluttered backgrounds, and lighting variations.
- **Silhouette ambiguity at low resolution:** At 28×28 resolution, fine-grained details such as stitching, fabric weave, buttons, and collars are suppressed. In particular, upper-body garment categories (*T-shirt/top*, *Shirt*, *Pullover*, *Coat*) present substantial geometric overlap in silhouette, forming an intrinsic bottleneck for linear and shallow classifiers.
- **Synthetic class balance:** Because the raw dataset is artificially curated to be perfectly uniform (10.0% per class), natural retail prevalence skew is absent. Performance degradation on specific

classes is therefore attributable strictly to visual ambiguity rather than class starvation.

1.3. Exploratory Data Analysis

This section covers the four required EDA items: input size, class distribution, imbalance analysis, and representative samples. We also study the pixel intensities, because they guide the preprocessing. To avoid data leakage, every statistic that uses pixel values is computed on the training split only (see Section 1.4 for the split).

1.3.1. Input size and data integrity

Every image in the 70,000-image official set has the same size: 28×28 pixels with one grayscale channel, stored as `uint8` values in $[0, 255]$. After `ToTensor`, each image becomes a $1 \times 28 \times 28$ float tensor with values in $[0, 1]$, which is 784 numbers per image.

We also checked that the image and label counts match, that all labels are between 0 and 9, and that there are no missing or non-finite values or shape inconsistencies were found.

1.3.2. Class distribution

1.3.2.1 Whole dataset (train and test).

Figure 1 shows the official split. Every class has exactly 6,000 training images and 1,000 test images.

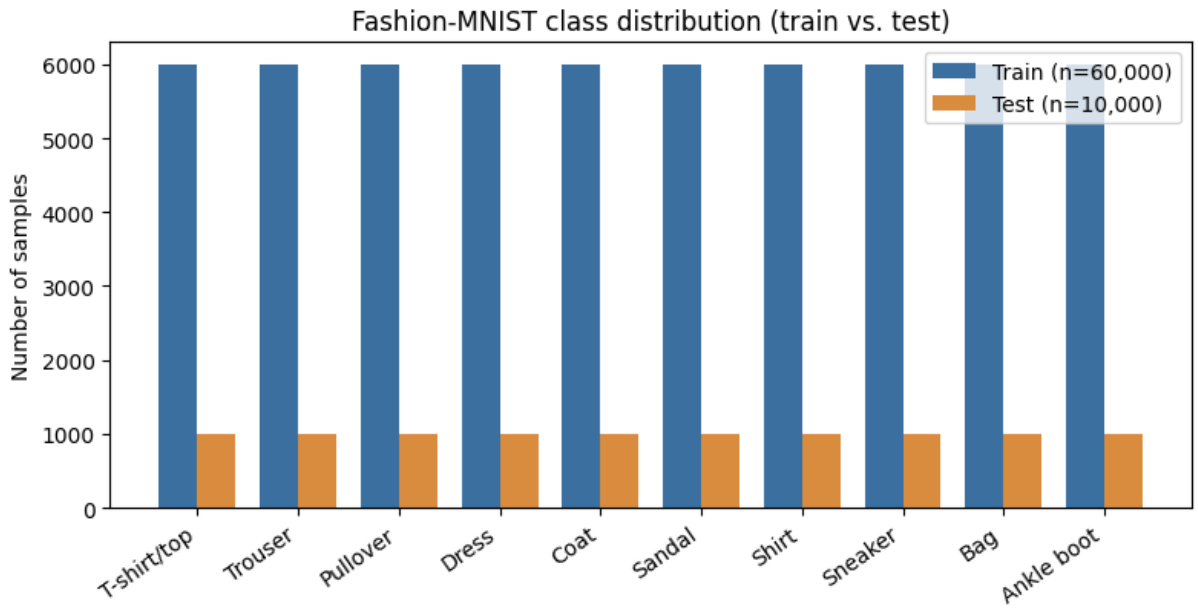


Figure 1: Number of images per class in the official training (60,000) and test (10,000) sets.

1.3.2.2 Training and validation splits.

The 60,000 official training images are split once into 80% train and 20% validation, using stratified sampling (class proportions are preserved in both subsets). We also use a fixed seed 42 so that the same

images are assigned to each subset in every run and the split can be exactly reproduced Figure 2 shows the result: 4,800 training and 1,200 validation images per class (48,000 and 12,000 in total). The 10,000 test images are left untouched.

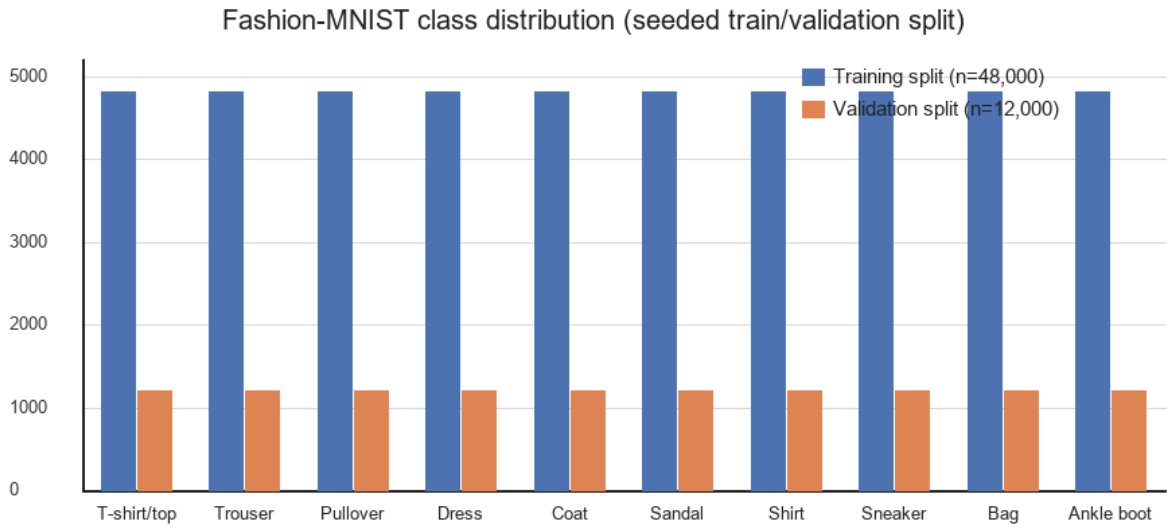


Figure 2: Images per class in the training split (48,000) and the validation split (12,000).

1.3.3. Imbalance analysis

Table 2 measures the imbalance in each partition. The ratio between the largest and the smallest class is 1.0 and the standard deviation of the class counts is 0, so the data is perfectly balanced in every partition.

Table 2: Class balance of each partition.

Partition	Images	Per class	Max/min ratio	Majority-class accuracy
Train (official)	60,000	6,000	1.0	10%
Test (official)	10,000	1,000	1.0	10%
Train (our split)	48,000	4,800	1.0	10%
Validation (ours)	12,000	1,200	1.0	10%

What this means for our experiments:

- A model that always predicts one class reaches only 10% accuracy, so Top-1 accuracy is a reliable main metric.
- No resampling or class weighting is needed. We train with the plain cross-entropy loss.
- We still report macro-F1, per-class metrics, and confusion matrices, to see which classes are confused with each other.
- A low score on one class (for example Shirt) is more likely caused by visual similarity with other classes than by a lack of examples.

1.3.4. Pixel-intensity distribution (training split only)

We compute the pixel mean and standard deviation over all pixels of the 48,000 training images only ($48{,}000 \times 28 \times 28 = 37{,}632{,}000$ pixels). Validation and test images are not used for these statistics to prevent any data leakage On the $[0, 255]$ scale:

$$\mu_{\text{raw}} = 72.99, \qquad \sigma_{\text{raw}} = 90.06 \quad \left(\sigma^2_{\text{raw}} \approx 8.11 \times 10^3\right). \tag{9}$$

After dividing by 255, these become $\mu \approx 0.2863$ and $\sigma \approx 0.3532$, the values used to normalize the image tensors in Section 1.4. Validation and test images are not used for these numbers, so there is no data leakage.

Figure 3 shows that about 50.2% of all pixels are 0 (the black background). The other pixels are spread over 1–255 and mostly belong to the garment.

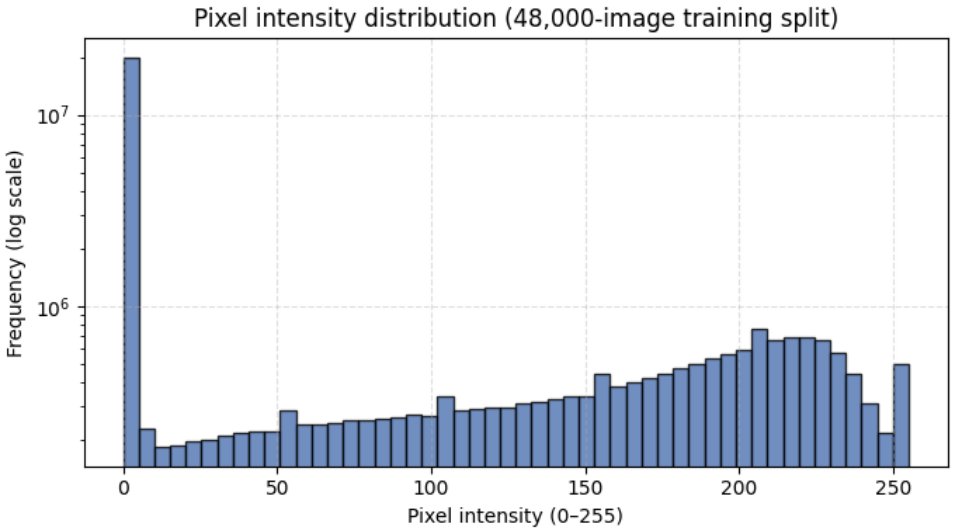


Figure 3: Pixel-intensity histogram (log scale) of the 48,000 training images. The tall bar at 0 is the background (about 50.2% of pixels).

Table 3 gives the mean intensity of each class. Since the background is black, a higher value tends to indicate that the item covers a larger part of the frame, but it is only a rough proxy: it also depends on how bright the item itself is (fabric colour, texture and contrast in the grayscale image), so a dark garment can have a low mean even when it fills the frame. Sandals and sneakers are the darkest, most likely because they leave a lot of empty space, while pullovers and coats are the brightest, most likely because they cover most of the frame. Mean intensity alone is therefore not enough to classify the images.

Table 3: Mean pixel intensity per class on the 48,000 training images.

Class	**Mean intensity** $[0, 255]$
Sandal	34.80
Sneaker	42.84
Trouser	56.99
Dress	66.12
Ankle boot	76.76
T-shirt/top	82.85
Shirt	84.83
Bag	90.17
Pullover	96.49
Coat	98.10
All classes	**72.99**

1.3.5. Representative samples

Figures 4 and 5 show seven random training images per class (seed 42).

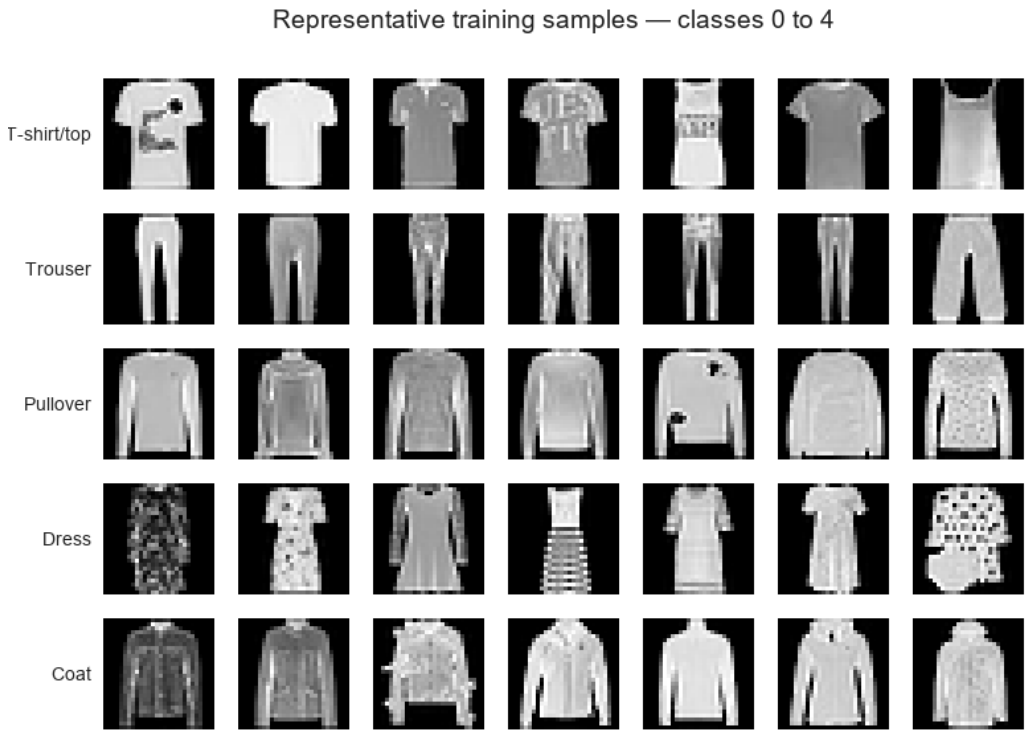


Figure 4: Random training images, classes 0–4 (T-shirt/top, Trouser, Pullover, Dress, Coat).

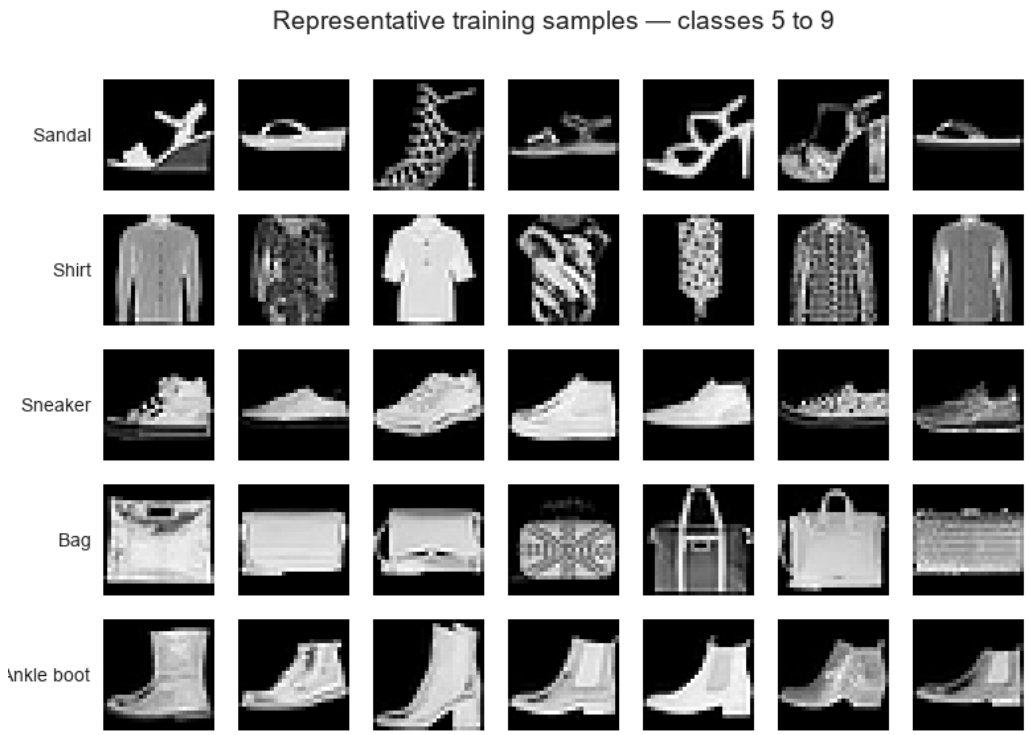


Figure 5: Random training images, classes 5–9 (Sandal, Shirt, Sneaker, Bag, Ankle boot).

We notice two things:

- Some classes vary a lot inside the same class. Bag includes handbags, backpacks, and clutches, so one class contains many different shapes and a single simple template per class is not enough.
- T-shirt/top, Shirt, Pullover, and *Coat* look alike at low resolution. We expect most classification errors to come from these four classes, and we check this in the error analysis.

1.3.6. Summary

- **Input size:** all images are $28 \times 28 \times 1$, so every model receives the same input.
- **Class distribution and imbalance:** perfectly balanced in all partitions, so accuracy is reliable and no rebalancing is needed.
- **Preprocessing:** about half of the pixels are black background, so we scale pixels to $[0, 1]$ and normalize with $\mu = 0.2863$ and $\sigma = 0.3532$, computed on the 48,000 training images only.
- **Difficult classes:** T-shirt/top, Shirt, Pullover, and Coat.
- **Leakage control:** validation images (12,000) are used for early stopping and checkpoint selection; test images (10,000) are used once for the final result.

1.4. Data splitting

1.4.1. Dataset Partitioning and Split Strategy

The complete Fashion-MNIST dataset comprises 70,000 grayscale images of size 28×28 distributed across 10 distinct categories (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot). We partition the data into three disjoint subsets combining official and custom splits:

- **Official Test Split:** We preserve the 10,000 test images provided by the official Fashion-MNIST benchmark through `torchvision.datasets.FashionMNIST` with `train=False`.
- **Custom Train / Validation Split:** The official 60,000-image training pool is divided into train (80%) and validation (20%) sets using `sklearn.model_selection.train_test_split`:
  - **Training set:** 48,000 images
  - **Validation set:** 12,000 images

Table 4: Dataset Partitioning and Class Distribution Summary

Subset	Total Samples	Samples / Class
Train (Custom 80%)	48,000	4,800
Validation (Custom 20%)	12,000	1,200
Test (Official)	10,000	1,000
Total	70,000	7,000

1.4.2. Seed and Deterministic Reproducibility

We distinguish between the seed used for data partitioning and the seeds used for training:

$$\text{SPLIT_SEED} = 42, \qquad \text{TRAIN_SEEDS} = \{42, 43, 44\}. \tag{10}$$

The fixed split seed produces the same stratified training and validation partitions for all models, variants, and runs. Each configuration is then trained independently from scratch using each of the three training seeds.

Before model initialization and training, the corresponding training seed is assigned to Python's random-number generator, NumPy, and the PyTorch CPU and CUDA generators. These seeds control stochastic operations such as parameter initialization, mini-batch shuffling, training augmentation, and dropout.

1.4.3. Stratification

Stratified sampling is applied when splitting the official 60,000 training images into the final training set (48,000 images) and validation set (12,000 images). Since the original 60,000 training images contain

exactly 6,000 samples per class, stratification maintains an exact uniform distribution in both sets:

- **Training set:** exactly 4,800 images for each class
- **Validation set:** exactly 1,200 images for each class

This ensures that neither the training set nor the validation set suffers from class imbalance or sampling bias.

1.4.4. Split Unit

The elementary split unit is a single independent image (one image = one sample). Each sample corresponds to an independently cataloged e-commerce product. There are no hierarchical groupings, multi-view captures, subject/patient identities, or temporal dependencies that could introduce inter-sample correlation.

1.4.5. Data Leakage Prevention

To prevent any statistical contamination or optimistic bias from the validation/test sets leaking into the training pipeline, the following measures are enforced:

1. **Split-Before-Processing Order:** Data splitting is performed before calculating any dataset-level statistics or applying transformations.
2. **Isolated Normalization Statistics:** The pixel mean (μ) and standard deviation (σ) are estimated exclusively from the 48,000 training images:

$$\mu = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} x_i, \quad \sigma = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (x_i - \mu)^2} \tag{11}$$

   These statistics are then applied to the validation and test sets for Z-score normalization. No validation or test pixels contribute to μ or σ.
3. **Data Augmentation Isolation:** Rotation, horizontal flipping,cropping are applied solely to the training split. Validation and test loaders apply only the Z-score normalization.
4. **Held-Out Test Isolation:** The official test set is completely isolated during training. It is never used for hyperparameter tuning, model selection, or early stopping decisions (which are driven purely by validation loss).

1.4.6. Subset Selection Rules

The final experiments will use the full FashionMNIST dataset. No class is intentionally excluded and no manually selected subset is planned for the final reported results.

1.5. Data Preprocessing

1.5.1. Data Cleaning and Invalid Annotation Handling

Before preprocessing, the pipeline checks image dimensions and image-label count consistency, pixel data types, and the validity of pixel values. Images must be grayscale arrays of size 28×28, with finite pixel values. Class labels must cover exactly the integers $\{0, \ldots, 9\}$.

These checks pass for the loaded dataset, so no samples are removed or manually corrected. If a check fails, an assertion stops execution rather than silently discarding or modifying the affected data. Labels are retained as integer class indices without one-hot encoding, as required by `CrossEntropyLoss`.

1.5.2. Input Normalization

All images already have the required spatial resolution of 28×28, so no resizing is applied. After augmented (applied to training images only), each image is converted to a tensor using `ToTensor`, which scales pixel intensities from $[0, 255]$ to $[0, 1]$.

The scaled intensities are standardized as

$$x_{\text{norm}} = \frac{x_{\text{raw}}/255 - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

where $\mu_{\text{train}} \approx 0.2863$ and $\sigma_{\text{train}} \approx 0.3532$ are computed from the 48,000 unaugmented training images only. The same statistics are used for training, validation, and testing.

No tokenization is used. Padding is applied only during the training crop augmentation described below; the final image size remains 28×28.

1.5.3. Data Augmentation

Training images undergo the following transformations in order:

1. Random rotation with an angle sampled from $[-10^\circ, 10^\circ]$.
2. Random horizontal flipping with probability 0.5.
3. Random cropping to 28×28 after the image is zero-padded by two pixels on all four sides (left, right, top and bottom).

These transformations are applied before tensor conversion and normalization, and the class label remains unchanged. Validation and test images undergo only tensor conversion and normalization, without random augmentation.

1.5.4. Input Tensor Shapes

Each preprocessed image has shape $(1, 28, 28)$, where the first dimension represents the grayscale channel. A batch therefore has image shape $(B, 1, 28, 28)$ and label shape $(B,)$.

The CNN directly receives the image tensor in this format. The Linear classifier and MLP flatten each image inside their forward passes, producing an input of shape $(B, 784)$.

1.5.5. Example Batch after Preprocessing

The notebook verifies one training batch with the following shapes:

```
Images: torch.Size([128, 1, 28, 28])
Labels: torch.Size([128])
```

Training uses a batch size of 128, with shuffled sample order. Validation and testing use a batch size of 256 without shuffling. The final batch may contain fewer samples because incomplete batches are retained.

2. Methodology

2.1. Pipeline Overview

The classification pipeline follows a standard sequential flow across all experiments. The overall process is represented by the following diagram:

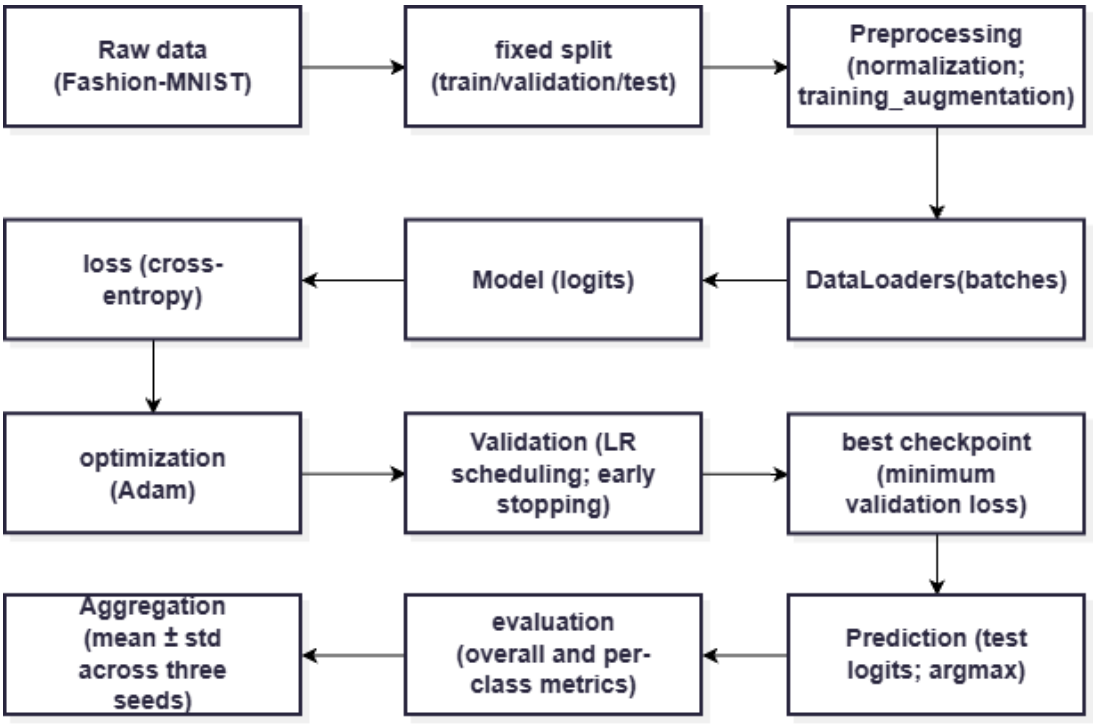


Figure 6: Pipeline Overview

Training and validation repeat until stopping. Each run restores its best checkpoint before test evaluation.

2.2. Baseline (Linear / softmax classifier)

2.2.1. Architecture and input representation

We use a linear classifier as the baseline for Fashion-MNIST classification. This model provides a low-complexity reference for assessing the benefits of nonlinear feature learning in the MLP and spatial feature extraction in the CNN row-sequence modeling in the LSTM, and patch-token modeling with self-attention in the Transformer.

Each preprocessed batch of grayscale images has shape $B \times 1 \times 28 \times 28$, where B denotes the batch size. The model flattens the channel and spatial dimensions while preserving the batch dimension, producing an input matrix $\mathbf{X} \in \mathbb{R}^{B \times 784}$. Each row contains the pixel values of one image in a fixed order. Flattening does not discard pixel values, but the classifier does not explicitly exploit two-dimensional neighborhoods.

The architecture consists of a single fully connected layer applied independently to every image in the batch:

$$\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{1}_B \mathbf{b}^T, \tag{12}$$

where $\mathbf{W} \in \mathbb{R}^{784\times 10}$, $\mathbf{b} \in \mathbb{R}^{10}$, and $\mathbf{1}_B \in \mathbb{R}^B$ is a vector of ones that broadcasts the bias across the batch. The output matrix $\mathbf{Z} \in \mathbb{R}^{B\times 10}$ contains one row per image and one logit per class.

There are no hidden layers, nonlinear hidden activations, or dropout layers in the forward pass. The baseline uses the same image preprocessing and data partitions as the other models.

2.2.2. Parameter count

The fully connected layer contains 784×10 weights and ten bias terms:

$$N_{\text{parameters}} = \underbrace{784 \times 10}_{\text{weights}} + \underbrace{10}_{\text{biases}} = \boxed{7{,}850}. \tag{13}$$

2.2.3. Strengths and limitations

2.2.3.a Strengths.

- **Computational efficiency:** Linear classifier has a simple architecture, making it fast to train and perform inference on Fashion-MNIST dataset.
- **Interpretability:** Its learned coefficients indicate how individual pixel intensities contribute to the predicted class, although interpretation is limited by correlations between pixels.
- **Natural multiclass extension:** Multinomial Logistic Regression with softmax can directly predict the probabilities of all 10 Fashion-MNIST classes.
- **Low model complexity:** Compared with deep neural networks, it has relatively few trainable parameters and a lower risk of overfitting, particularly when regularization is applied.
- **Strong baseline:** It provides a simple baseline for evaluating whether more complex models, such as CNNs, deliver meaningful performance improvements.

2.2.3.b Limitations.

- **Linear decision boundaries:** Softmax Regression learns linear decision boundaries in the 784-dimensional pixel space, limiting its ability to distinguish classes with complex, nonlinear patterns.
- **Limited spatial feature extraction:** Flattening each 28×28 image into a 784-dimensional vector does not explicitly capture local spatial relationships, such as edges, textures, and shapes.

- **Sensitivity to image variations:** Changes in object position, shape, or orientation can significantly alter pixel values. Softmax regression Regression does not inherently provide translation or rotation invariance.
- **Performance ceiling:** Although computationally efficient, it may underfit the visual complexity of Fashion-MNIST and generally lacks the representational capacity of CNN-based models.

2.2.4. Self-implemented components and library dependencies

Self-implemented: `LinearClassifier` class wrapper and forward input-flattening flow.

Library: `torch.nn.Linear`, `torch.nn.CrossEntropyLoss`.

2.3. Multilayer Perceptron (MLP)

2.3.1. Architecture and input representation.

The model is a deep feedforward network with three hidden dense layers. Like the linear baseline, it receives a batch of preprocessed grayscale images with shape $B \times 1 \times 28 \times 28$, where B is the batch size. Each image is flattened independently, producing an input matrix $\mathbf{X} \in \mathbb{R}^{B\times 784}$. Each row contains the pixel values of one image in a fixed order.

The layer sequence is defined as follows:

- **Hidden layer 1:** Dense layer with 784 inputs and 256 outputs $\rightarrow$ `ReLU` $\rightarrow$ dropout with probability $p = 0.3$, producing an output of shape $B \times 256$.
- **Hidden layer 2:** Dense layer with 256 inputs and 128 outputs $\rightarrow$ `ReLU` $\rightarrow$ dropout with probability $p = 0.3$, producing an output of shape $B \times 128$.
- **Hidden layer 3:** Dense layer with 128 inputs and 64 outputs $\rightarrow$ `ReLU` $\rightarrow$ dropout with probability $p = 0.3$, producing an output of shape $B \times 64$.
- **Output layer:** A dense layer maps 64 inputs to ten raw logits per image, with output shape $B \times 10$.

Writing $\mathbf{H}^{(0)} = \mathbf{X}$, the hidden representations are computed as

$$\mathbf{H}^{(t)} = \mathrm{Dropout}_{0.3}\Big(\mathrm{ReLU}\Big(\mathbf{H}^{(t-1)}\mathbf{W}^{(t)} + \mathbf{1}_B\big(\mathbf{b}^{(t)}\big)^T\Big)\Big), \qquad t = 1,2,3, \tag{14}$$

where $\mathbf{W}^{(t)} \in \mathbb{R}^{d_{t-1}\times d_t}$, $\mathbf{b}^{(t)} \in \mathbb{R}^{d_t}$, and $(d_0,d_1,d_2,d_3,d_4) = (784,256,128,64,10)$. The vector $\mathbf{1}_B \in \mathbb{R}^B$ broadcasts the bias across the batch. The output logits are

$$\mathbf{Z} = \mathbf{H}^{(3)}\mathbf{W}^{(4)} + \mathbf{1}_B\big(\mathbf{b}^{(4)}\big)^T \in \mathbb{R}^{B\times 10}. \tag{15}$$

Each row of $\mathbf{Z}$ contains the unnormalized class logits for one image. No softmax is applied in the forward pass because `CrossEntropyLoss` incorporates log-softmax internally. The network uses dropout as its only explicit model regularizer and contains no batch normalization.

Activation function: ReLU, defined as $\mathrm{ReLU}(u) = \max(0, u)$, is applied elementwise after each hidden dense layer. It provides the nonlinearity needed to learn complex feature relationships. Without nonlinear activations, the composition of the dense layers would reduce to a single affine transformation.

Regularization: During training, dropout with $p = 0.3$ is applied after the ReLU of each hidden layer. It independently sets hidden activations to zero with probability 0.3 and scales the retained activations by $1/(1-p)$, discouraging co-adaptation between hidden units. Dropout is disabled during evaluation.

2.3.2. Parameter Budget Analysis.

The network comprises exactly 242,762 trainable parameters, broken down in Table 5. Only the `Linear` layers carry trainable parameters; ReLU and dropout have none.

Table 5: Layer-by-layer Trainable Parameter Breakdown of the `MLP`

Block / Layer	Configuration	Parameter Formula	Count
Hidden 1	$\mathrm{Linear}(784 \to 256)$	$(784 \times 256) + 256$	200,960
Hidden 2	$\mathrm{Linear}(256 \to 128)$	$(256 \times 128) + 128$	32,896
Hidden 3	$\mathrm{Linear}(128 \to 64)$	$(128 \times 64) + 64$	8,256
Output	$\mathrm{Linear}(64 \to 10)$	$(64 \times 10) + 10$	650
Total Trainable			**242,762**

2.3.3. Strengths and limitations

2.3.3.a Strengths

1. **Non-linear modelling:** With hidden layers and ReLU activations, the MLP can learn non-linear interactions between pixels and non-linear decision boundaries, which the linear baseline cannot represent. This is particularly useful for visually similar classes such as shirts, T-shirts, and pullovers.

2. **Learned hierarchical features:** Each hidden layer re-represents the output of the previous one, so the network learns its own intermediate features from raw pixels instead of relying on hand-crafted ones.

3. **End-to-end training:** All weights are optimized jointly by backpropagation, which makes training straightforward and scalable to large datasets.

4. **Flexible architecture:** The number of hidden layers and units can be chosen freely to trade off capacity, speed, and generalization. The model has 242,762 parameters and a very simple structure, so it is cheap to train and fast at inference.

5. **Controllable generalization:** Its capacity can be regularized with dropout, and with data augmentation and early stopping, to limit overfitting.

2.3.3.b Limitations

1. **No spatial inductive bias:** The image is flattened into a 784-dimensional vector and every pixel is connected to every hidden unit, so neighbouring pixels are not treated differently from distant ones. The model would behave identically if all images were shuffled by the same fixed pixel permutation, and it does not capture local patterns such as edges and textures.

2. **Position-specific weights and limited spatial reuse:** The first fully connected layer contains 200,960 of the model's 242,762 parameters ($\approx 83\%$), with weights associated with specific input pixel positions rather than shared across locations. Consequently, shifting an image can substantially change hidden activations, and similar patterns appearing at different positions are not explicitly handled by shared detectors as in a CNN. This can limit parameter efficiency when learning repeated spatial patterns. Robustness to translations must be learned from data, for example through augmentation, rather than being explicitly built into the architecture.

3. **Low interpretability:** Unlike the linear baseline, the weights of the hidden layers do not map directly to pixel-level evidence for a class, so the model's decisions are harder to explain.

2.3.4. Self-implemented components and library dependencies

Self-implemented: the `MLP` class, the dynamic layer construction from `hidden_dims`, and the dropout regularization setup.

Library components:

`torch.nn.Linear`, `torch.nn.ReLU`,
`torch.nn.Dropout`, `torch.nn.Sequential`, and
`torch.nn.CrossEntropyLoss` from PyTorch.

2.4. Convolutional Neural Network (CNN)

2.4.1. Architecture and input representation

The CNN uses a custom architecture with three convolutional blocks (`SimpleCNN`, using no pretrained backbone) structured into a feature extractor and a classifier head:

- **Feature Extraction Stage:**

  - **Block 1:** A 3×3 convolution maps 1 input channel to 64 output channels (padding 1), followed by batch normalization, ReLU, 2-by-2 max pooling, and 0.25 dropout. It downsamples the feature map from 28×28 to 14×14.
  - **Block 2:** A 3×3 convolution maps 64 input channels to 128 output channels (padding 1), followed by batch normalization, ReLU, 2-by-2 max pooling, and 0.25 dropout. It downsamples the feature map from 14×14 to 7×7.
  - **Block 3:** A 3×3 convolution maps 128 channels to 128 channels (padding 1), followed by batch normalization, ReLU, and 0.3 dropout. It operates on 7×7 feature maps.

- **Classification Stage (`classifier`):** The classifier applies global average pooling across the spatial dimensions of each feature map and flattening, then a 128-to-64 linear layer with ReLU and 0.3 dropout, followed by a 64-to-10 output layer.

The input is a batch of preprocessed grayscale images with shape $B \times 1 \times 28 \times 28$, where B is the batch size. Unlike the MLP, the CNN retains explicit spatial dimensions throughout its feature extractor.

Convolution: Each convolutional filter operates on a local 3×3 neighborhood across all input channels. For C_{in} input channels, a filter therefore has shape $C_{\text{in}} \times 3 \times 3$. At each spatial position, the filter computes a weighted sum of the neighborhood values and adds a bias. The same weights are reused across spatial positions, allowing a learned pattern detector to respond at different locations. Stride 1 and padding 1 preserve the input height and width before pooling.

Feature maps: The spatial responses of one convolutional filter form one output feature map. Multiple filters produce multiple output channels, which can learn different patterns, such as edges, local intensity changes, and combinations of garment features. Batch normalization and ReLU then transform these responses. After Blocks 1, 2, and 3, the feature tensors have shapes $B \times 64 \times 14 \times 14$, $B \times 128 \times 7 \times 7$, and $B \times 128 \times 7 \times 7$, respectively. Each spatial position retains information associated with a region of the input image.

Max pooling: The first two blocks apply 2×2 max pooling with stride 2 independently to each channel. Each non-overlapping 2×2 region is replaced by its maximum activation, retaining the strongest response within that region. This halves the height and width while preserving the number of channels, reducing the size of the feature tensors passed to subsequent layers. Max pooling introduces no trainable parameters.

Global average pooling: After Block 3, global average pooling computes one average value for each

feature map:

$$g_{n,c} = \frac{1}{49} \sum_{i=1}^{7} \sum_{j=1}^{7} A_{n,c,i,j}, \tag{16}$$

where $\mathbf{A}$ is the final feature tensor, n indexes images, and c indexes channels. This operation transforms $B \times 128 \times 7 \times 7$ into $B \times 128 \times 1 \times 1$. Flattening produces a $B \times 128$ matrix for the dense classifier, whose final output has shape $B \times 10$. Global average pooling has no trainable parameters and summarizes each channel without preserving its individual spatial positions.

2.4.2. Parameter Budget Analysis.

To enable rigorous benchmarking against the MLP architecture under a matched parameter capacity target ($\sim$ 230K–245K), the network comprises exactly 231,626 trainable parameters, broken down in Table 6.

Table 6: Layer-by-layer Trainable Parameter Breakdown of `SimpleCNN`

Block / Layer	Configuration	Parameter Formula	Count
Block 1	$\text{Conv2d}(1 \to 64, 3 \times 3, \text{pad} = 1)$	$(64 \times 1 \times 3 \times 3) + 64$	640
	$\text{BatchNorm2d}(64)$	$\gamma(64) + \beta(64)$	128
Block 2	$\text{Conv2d}(64 \to 128, 3 \times 3, \text{pad} = 1)$	$(128 \times 64 \times 3 \times 3) + 128$	73,856
	$\text{BatchNorm2d}(128)$	$\gamma(128) + \beta(128)$	256
Block 3	$\text{Conv2d}(128 \to 128, 3 \times 3, \text{pad} = 1)$	$(128 \times 128 \times 3 \times 3) + 128$	147,584
	$\text{BatchNorm2d}(128)$	$\gamma(128) + \beta(128)$	256
Classifier	$\text{Linear}(128 \to 64)$	$(64 \times 128) + 64$	8,256
	$\text{Linear}(64 \to 10)$	$(10 \times 64) + 10$	650
Total Trainable			**231,626**

2.4.3. Strengths and limitations

2.4.3.a Strengths

- **Local connectivity and weight sharing:** Each 3×3 convolutional filter operates on a local neighborhood and shares its weights across spatial positions in the input feature maps. This allows the same pattern detector to be reused at different locations and exploits the local spatial structure of Fashion-MNIST images. For fixed kernel size and channel counts, the number of convolutional parameters is independent of the input's spatial dimensions.
- **Hierarchical feature learning:** Stacked convolutional layers build features from simple to complex: early layers respond to edges and contours, while deeper layers combine them into larger shape parts such as sleeves, collars, straps, or soles, which helps to separate similar garment categories.
- **Robustness to small shifts of the item:** The same filters scan the whole image, so a pattern such as an edge or a collar is detected wherever it appears. If the item moves by a few pixels, the detected

patterns simply move with it. Max pooling further reduces the effect of small shifts, because it keeps only the strongest response in each small region. This suits the small shifts introduced by the random-crop augmentation.

- **Parameter efficiency:** Using global average pooling in place of a large fully connected layer keeps the model compact (231,626 parameters) and independent of the spatial size of the feature map.
- **Adaptability:** The depth, number of channels, and kernel size can be changed easily to fit the data, so the same design can be reused for other image datasets.

2.4.3.b Limitations

- **High computational and memory cost** In a convolutional layer, the same filters are applied at every spatial position and across many channels, so the number of operations is much larger than the number of weights. A CNN also produces many feature maps at high spatial resolution, and during training these intermediate activations must be stored for backpropagation. Its computation and memory use therefore depend on the size of the feature maps and the batch size, not only on the number of parameters, and they grow as more convolutional layers are stacked.
- **Limited receptive field:** With three 3×3 convolutions and two pooling layers, each output unit sees only about 18×18 pixels of the input, so the model captures global context only indirectly, through pooling and averaging.
- **Loss of spatial detail:** Max pooling and global average pooling discard exact position information, which can hide fine differences between visually similar classes such as Shirt, T-shirt/top, Pullover, and Coat.
- **Limited built-in invariance:** Only translation is handled structurally; robustness to rotation, scale, or flipping is not guaranteed by the architecture and must be learned from data, for example through augmentation.
- **Low interpretability:** Unlike the linear baseline, the learned filters and feature maps do not map directly to pixel-level evidence for a class, so the model's predictions are harder to explain.

2.4.4. Self-implemented components and library dependencies

Self-implemented: Complete custom ConvNet architecture (`SimpleCNN`, no pretrained backbone).

Library components: Convolution, batch normalization, pooling, dropout, adaptive average pooling, flattening, and linear layers from PyTorch.

2.5. Long Short-Term Memory (LSTM)

2.5.1. Architecture Description and input representation

The LSTM model (`RowLSTM`) interprets each image as a sequence of pixel rows. A two-layer recurrent network produces an image representation, which is passed to a small classifier head.

Timestep definition: The input batch $\mathbf{X} \in \mathbb{R}^{B\times 1\times 28\times 28}$ is reshaped to $\mathbf{X}_{\text{seq}} \in \mathbb{R}^{B\times 28\times 28}$ by removing the singleton channel dimension. The first dimension indexes images, the second indexes timesteps, and the third contains the pixel values within each row. Each image is processed from top to bottom over $T = 28$ timesteps. Timestep t corresponds to image row t, rather than to an individual pixel. This is a sequential representation of a static image, not a temporal sequence.

Input size: At each timestep, the first LSTM layer receives one image row containing 28 pixel values per image. Its input at that timestep therefore has shape $B \times 28$, where B is the batch size. Across all 28 rows, the complete input sequence has shape $B \times 28 \times 28$.

The first layer produces a 132-dimensional hidden representation at each timestep, forming an output sequence of shape $B \times 28 \times 132$. Dropout is applied to this sequence between the two layers during training without changing its shape.

At timestep t, the second LSTM layer receives the first layer's hidden representation from the same timestep. Its input therefore has shape $B \times 132$ per timestep and $B \times 28 \times 132$ for the complete sequence. Both layers process 28 timesteps, corresponding to the 28 image rows. The sequence length remains unchanged; only the input feature dimension changes from 28 in the first layer to 132 in the second layer.

The architecture consists of the following stages:

- **Recurrent Stage:** A two-layer unidirectional LSTM (`nn.LSTM`) with hidden size 132 processes the row sequences. The first layer receives inputs of shape $B \times 28$, while the second receives the first layer's hidden representations of shape $B \times 132$, with inter-layer dropout of $p = 0.3$ during training.

  For either layer, let d denote its input size, where $d = 28$ for the first layer and $d = 132$ for the second. At timestep t, the current input $\mathbf{X}_t \in \mathbb{R}^{B\times d}$ is concatenated with the previous hidden state $\mathbf{H}_{t-1} \in \mathbb{R}^{B\times 132}$:

  $$\mathbf{S}_t = [\mathbf{H}_{t-1} \parallel \mathbf{X}_t] \in \mathbb{R}^{B\times(132+d)}, \tag{17}$$

  where $\parallel$ denotes concatenation along the feature dimension. Therefore, $\mathbf{S}_t$ has shape $B \times 160$ in the first layer and $B \times 264$ in the second.

the gates and state updates are

$$
\begin{aligned}
\mathbf{I}_t &= \sigma\left(\mathbf{S}_t\mathbf{W}_i + \mathbf{1}_B\mathbf{b}_i^T\right), \\
\mathbf{F}_t &= \sigma\left(\mathbf{S}_t\mathbf{W}_f + \mathbf{1}_B\mathbf{b}_f^T\right), \\
\mathbf{O}_t &= \sigma\left(\mathbf{S}_t\mathbf{W}_o + \mathbf{1}_B\mathbf{b}_o^T\right), \\
\mathbf{G}_t &= \tanh\left(\mathbf{S}_t\mathbf{W}_g + \mathbf{1}_B\mathbf{b}_g^T\right), \\
\mathbf{C}_t &= \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \mathbf{G}_t, \\
\mathbf{H}_t &= \mathbf{O}_t \odot \tanh(\mathbf{C}_t).
\end{aligned}
\tag{18}
$$

Here, $\mathbf{I}_t$, $\mathbf{F}_t$, and $\mathbf{O}_t$ are the input, forget, and output gates, respectively. $\mathbf{G}_t$ is the candidate cell content, $\mathbf{C}_t$ is the cell state, and $\mathbf{H}_t$ is the hidden state. All have shape $B \times 132$, with one row per image and 132 features per row.

Elementwise activation and gating: The logistic sigmoid, $\sigma(u) = 1/(1+e^{-u})$, is applied separately to every element of each gate's affine output. Each image therefore has 132 gate values, each lying between 0 and 1. These values control the corresponding memory features individually rather than applying one shared gate value to the entire hidden vector.

A value close to 0 suppresses the corresponding feature, a value close to 1 allows most of it to pass, and intermediate values provide partial transmission. The gates are continuous rather than binary, and their values are not required to sum to 1. Although gating is elementwise, each gate value is computed from the concatenated input $\mathbf{S}_t$ and can depend on all its features.

The components have the following roles:

- **Forget gate $\mathbf{F}_t$:** Controls how much of the previous cell state $\mathbf{C}_{t-1}$ is retained. In $\mathbf{F}_t \odot \mathbf{C}_{t-1}$, values near 0 discard previous content, while values near 1 preserve it.
- **Candidate content $\mathbf{G}_t$:** Proposes new memory content using the current input and previous hidden state. The elementwise tanh activation maps each candidate value to the range $(-1, 1)$, allowing both positive and negative updates. Unlike the sigmoid gates, $\mathbf{G}_t$ represents content rather than an amount to retain or transmit.
- **Input gate $\mathbf{I}_t$:** Controls how much candidate content is written into the cell state. The product $\mathbf{I}_t \odot \mathbf{G}_t$ selectively scales the proposed update.
- **Output gate $\mathbf{O}_t$:** Controls how much of the transformed cell state $\tanh(\mathbf{C}_t)$ is exposed through the hidden state $\mathbf{H}_t$. It regulates the output without directly changing the stored cell state.

The cell-state update combines retained previous content with selected new content: $\mathbf{F}_t \odot \mathbf{C}_{t-1}$

provides the retained memory, while $\mathbf{I}_t \odot \mathbf{G}_t$ provides the new contribution. The hidden state then exposes a gated, bounded transformation of this updated memory.

Both tanh and $\odot$ operate elementwise. In particular, $[\mathbf{A} \odot \mathbf{B}]_{b,j} = A_{b,j}B_{b,j}$, so each gate scales the matching feature of the same image without mixing features or samples during this multiplication.

For each gate or candidate update,

$$\mathbf{W}_\bullet \in \mathbb{R}^{(132+d)\times 132}, \qquad \mathbf{b}_\bullet \in \mathbb{R}^{132}.$$

The column vector $\mathbf{1}_B \in \mathbb{R}^{B\times 1}$ broadcasts the bias across the batch. Each layer and gate has its own parameters, which are shared across timesteps and images.

- **Classification Stage (`classifier`):** The final hidden-state matrix of the second layer, $\mathbf{H}_{28}^{(2)} \in \mathbb{R}^{B\times 132}$, contains one image representation per row. It passes through dropout with $p = 0.3$, a 132-to-64 linear layer, ReLU, dropout with $p = 0.3$, and a 64-to-10 output layer. The intermediate dense representation has shape $B \times 64$, and the final logit matrix has shape $\mathbf{Z} \in \mathbb{R}^{B\times 10}$.

Hidden representation: Each recurrent layer maintains hidden and cell states with shape $B \times 132$. Because the network is unidirectional, its representation at timestep t summarizes information from rows 1 through t without accessing later rows. The second layer produces a hidden-state sequence with shape $B \times 28 \times 132$.

The classifier uses only the final hidden state of the second layer, $\mathbf{H}_{28}^{(2)} \in \mathbb{R}^{B\times 132}$. Each row of this matrix is a learned 132-dimensional summary of one image after all 28 rows have been processed. The model does not flatten or average the entire hidden-state sequence for classification.

2.5.2. Parameter Budget Analysis.

To keep a comparable capacity with the MLP and CNN ($\sim$ 230K–245K parameters), the network has exactly 235,146 trainable parameters, broken down in Table 7. A recurrent layer with input size d and hidden size h has $4h(d+h)+8h$ parameters: four gate/candidate transformations, each with input weights, recurrent weights, and two bias vectors (`bias_ih` and `bias_hh` in PyTorch).

Table 7: Layer-by-layer Trainable Parameter Breakdown of `RowLSTM`

Block / Layer	Configuration	Parameter Formula	Count
Recurrent	LSTM layer $1(28 \to 132)$	$4 \cdot 132 \cdot (28 + 132) + 2 \cdot 4 \cdot 132$	85,536
	LSTM layer $2(132 \to 132)$	$4 \cdot 132 \cdot (132 + 132) + 2 \cdot 4 \cdot 132$	140,448
Classifier	Linear$(132 \to 64)$	$(64 \times 132) + 64$	8,512
	Linear$(64 \to 10)$	$(10 \times 64) + 10$	650
Total Trainable			**235,146**

2.5.3. Strengths and limitations

2.5.3.a Strengths

- **Sequential memory of the image:** The hidden state carries information from the rows already read, so the model can relate the parts of an item along the vertical direction (for example neckline, body, and hem of a garment) instead of treating each row independently.
- **Gating against vanishing gradients:** The forget, input, and output gates and the additive cell-state update let information and gradients flow across many timesteps more easily than in a plain recurrent network.
- **Weight sharing over time:** The same weights are used for all 28 rows, so the number of parameters does not depend on the sequence length, and the model remains compact.
- **Simple input representation:** Each row is directly a 28-dimensional feature vector, so no patching or embedding step is needed, and the whole image is compressed into a fixed-size hidden state.

2.5.3.b Limitations

- **Sequential computation:** Each timestep depends on the previous hidden state, so the 28 steps cannot be processed in parallel, which makes training and inference less GPU-friendly than convolutions or attention.
- **Weak two-dimensional structure:** The image is forced into a one-dimensional order of rows. Vertical neighbours are linked only through the hidden state, and horizontal neighbours inside a row are mixed by dense weights without explicit local structure.
- **Information bottleneck:** Only the final hidden state of the top layer is used, so a 132-dimensional vector must summarize the whole image, and details from early rows may be diluted over the 28 steps.

- **One-directional context:** Rows are read only from top to bottom, so early rows are processed without knowledge of the later ones.
- **Limited built-in invariance:** Shifting or rotating an item changes the content of many rows, and the model has no structural mechanism to handle this, so it must rely on data and augmentation..
- **Low interpretability:** The hidden and cell states are distributed vectors that do not map directly to pixel-level evidence for a class.

2.5.4. Self-implemented components and library dependencies

Self-implemented: Row-sequence formulation of the image, selection of the final hidden state, and the classifier head (`RowLSTM`, no pretrained backbone).

Library components: The recurrent layers (`nn.LSTM`), dropout, ReLU, and linear layers from PyTorch.

2.6. Transformer

2.6.1. Architecture Description and Input representation

The Transformer model (`PatchTransformer`) follows a Vision-Transformer design: the image is split into patches, each patch becomes a token, and a stack of self-attention layers lets all tokens exchange information before classification:

- **Tokenization Stage:** The image is divided into non-overlapping 4×4 patches, giving $7 \times 7 = 49$ patches. Each patch is flattened into a 16-dimensional vector and mapped to a 64-dimensional *token embedding* by a shared linear projection. A learnable `[CLS]` token is prepended, producing 50 tokens in total.

  The shared projection encodes patch content without explicitly identifying its location. Self-attention alone also does not encode token order or two-dimensional patch coordinates. Consequently, patch content alone does not tell the model how the patches are arranged within the original image. A learnable *positional embedding* $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{50\times 64}$ is therefore added to the token embeddings. Each sequence position has its own learned positional vector, allowing the model to distinguish patch locations and learn relationships that depend on their spatial arrangement. Dropout with probability 0.1 is applied to the resulting embeddings.

- **Encoder Stage:** The token embeddings pass through six Transformer encoder layers with the same structure but separate trainable parameters. Each layer contains multi-head self-attention (MHA) and a position-wise feed-forward network (FFN), with pre-normalization, dropout, and residual connections. The input and output of each layer have shape $B \times 50 \times 64$.

  Structure of one encoder layer: Let $\mathbf{Z} \in \mathbb{R}^{B\times 50\times 64}$ denote the input to one layer. Layer normal-

ization is applied before each sublayer, giving

$$\mathbf{U}_{\text{attn}} = \text{LN}_1(\mathbf{Z}), \qquad \mathbf{U}_{\text{ffn}} = \text{LN}_2(\mathbf{Z}'). \tag{19}$$

The layer then computes

$$\mathbf{Z}' = \mathbf{Z} + \text{Dropout}_{0.1}(\text{MHA}(\mathbf{U}_{\text{attn}})), \tag{20}$$

$$\mathbf{Z}'' = \mathbf{Z}' + \text{Dropout}_{0.1}(\text{FFN}(\mathbf{U}_{\text{ffn}})). \tag{21}$$

This is a pre-norm configuration: LN_1 normalizes the input to attention, while LN_2 normalizes the updated representation before the FFN. Each normalization operates across the 64 features of each token and has its own learnable parameters.

The additions implement residual connections, combining each sublayer's output with its input. They preserve a direct path for information and help gradients propagate through the encoder. Dropout is applied to each sublayer's output before the residual addition. Both sublayers return 64-dimensional token representations, allowing these elementwise additions without changing the tensor shape.

Multi-head self-attention (MHA): MHA allows each token to combine information from all 50 tokens within the same image. The model uses four attention heads, each with feature dimension $64/4 = 16$. The computation has three steps.

First, each head h projects the normalized input into queries, keys, and values:

$$\begin{aligned} \mathbf{Q}_h &= \mathbf{U}_{\text{attn}}\mathbf{W}_h^Q + \mathbf{b}_h^Q, \\ \mathbf{K}_h &= \mathbf{U}_{\text{attn}}\mathbf{W}_h^K + \mathbf{b}_h^K, \\ \mathbf{V}_h &= \mathbf{U}_{\text{attn}}\mathbf{W}_h^V + \mathbf{b}_h^V, \end{aligned} \tag{22}$$

where $\mathbf{W}_h^Q, \mathbf{W}_h^K, \mathbf{W}_h^V \in \mathbb{R}^{64\times 16}$ and $\mathbf{b}_h^Q, \mathbf{b}_h^K, \mathbf{b}_h^V \in \mathbb{R}^{16}$. Each bias vector is broadcast across the batch and token dimensions. Each projected tensor has shape $B \times 50 \times 16$. Queries and keys determine which tokens are relevant to one another, while values provide the information to be combined.

Second, each head computes attention weights and uses them to aggregate values:

$$\begin{aligned} \mathbf{A}_h &= \text{softmax}\left(\frac{\mathbf{Q}_h\mathbf{K}_h^T}{\sqrt{16}}\right), \\ \mathbf{H}_h &= \text{Dropout}_{0.1}(\mathbf{A}_h)\mathbf{V}_h. \end{aligned} \tag{23}$$

The operations are performed separately for each image, and the transpose applies to the final two dimensions. The attention matrix $\mathbf{A}_h \in \mathbb{R}^{B\times 50\times 50}$ contains one row for each receiving token and one column for each source token. Softmax is applied across source tokens, so each row specifies their relative contributions before attention dropout. The scaling factor $\sqrt{16}$ controls the magnitude of the query–key dot products.

Third, the four head outputs are concatenated along the feature dimension and projected back to 64 features:

$$\mathrm{MHA}(\mathbf{U}_{\mathrm{attn}}) = \mathrm{Concat}(\mathbf{H}_1, \mathbf{H}_2, \mathbf{H}_3, \mathbf{H}_4)\,\mathbf{W}^O + \mathbf{b}^O, \tag{24}$$

where $\mathbf{W}^O \in \mathbb{R}^{64\times 64}$ and $\mathbf{b}^O \in \mathbb{R}^{64}$. The resulting tensor has shape $B \times 50 \times 64$. The heads can learn different relationships among image regions, although their roles are not specified in advance.

Feed-forward network (FFN): After attention exchanges information between tokens, the FFN transforms the features of each token independently:

$$\mathrm{FFN}(\mathbf{U}_{\mathrm{ffn}}) = \mathrm{Dropout}_{0.1}(\mathrm{GELU}(\mathbf{U}_{\mathrm{ffn}}\mathbf{W}_1 + \mathbf{b}_1))\,\mathbf{W}_2 + \mathbf{b}_2, \tag{25}$$

where $\mathbf{W}_1 \in \mathbb{R}^{64\times 160}$, $\mathbf{b}_1 \in \mathbb{R}^{160}$, $\mathbf{W}_2 \in \mathbb{R}^{160\times 64}$, and $\mathbf{b}_2 \in \mathbb{R}^{64}$. Bias vectors are broadcast across the batch and token dimensions.

Each token is expanded from 64 to 160 features, transformed by GELU and dropout, and projected back to 64 features. The same FFN parameters are shared across all token positions. Unlike MHA, the FFN does not directly combine information from different tokens. Its internal dropout follows GELU, while the separate output dropout in Equation 21 is applied before the residual addition. All dropout operations are disabled during evaluation.

Encoder output: The output $\mathbf{Z}''$ of each layer becomes the input to the next layer. After six layers, the encoder returns a tensor of shape $B \times 50 \times 64$.

- **Classification Stage:** After the sixth encoder layer, the model extracts the `[CLS]` token representation, giving a tensor of shape $B \times 64$. Through self-attention, this token can aggregate information from the patch tokens and learn an image-level representation. It passes through a final layer normalization and a 64-to-10 linear layer:

$$\mathbf{L} = \mathrm{LN}_{\mathrm{final}}\left(\mathbf{Z}^{(6)}_{[:,0,:]}\right)\mathbf{W}_{\mathrm{cls}} + \mathbf{b}_{\mathrm{cls}} \in \mathbb{R}^{B\times 10}. \tag{26}$$

Each row contains ten raw class logits for one image. No softmax is applied in the forward pass because `CrossEntropyLoss` incorporates log-softmax internally.

Attention inputs and outputs. In each head, the input token matrix $\mathbf{Z}$ is linearly projected into queries $\mathbf{Q}$, keys $\mathbf{K}$, and values $\mathbf{V}$ (each 50×16). The attention weights

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{50\times 50} \tag{27}$$

state how strongly each token attends to every other token, and the output $\mathbf{AV}$ has the same number of tokens as the input, with each token now containing information from the tokens it attends to. The outputs of the four heads are concatenated and projected back to 64 dimensions.

The input batch $\mathbf{X} \in \mathbb{R}^{B\times 1\times 28\times 28}$ is cut into patches with tensor unfolding and reshaped to $\mathbb{R}^{B\times 49\times 16}$ (49 tokens, each a flattened 4×4 patch). After projection, the `[CLS]` token and positional embedding, the encoder receives $\mathbb{R}^{B\times 50\times 64}$. Pixels use the same normalization as the other models ($\mu = 0.2863$, $\sigma = 0.3532$, computed from the training set only).

2.6.2. Parameter Budget Analysis.

To keep a comparable capacity with the MLP and CNN ($\sim$ 230K–245K parameters), the network has exactly 230,730 trainable parameters, broken down in Table 8.

Table 8: Layer-by-layer Trainable Parameter Breakdown of `PatchTransformer`

Block / Layer	Configuration	Parameter Formula	Count
Tokenization	Linear$(16 \to 64)$ (patch projection)	$(64 \times 16) + 64$	1,088
	`[CLS]` token	1×64	64
	Positional embedding	50×64	3,200
Encoder (per layer)	Multi-head attention (Q, K, V, output proj.)	$(3 \cdot 64 \cdot 64 + 192) + (64 \cdot 64 + 64)$	16,640
	Linear$(64 \to 160)$	$(160 \times 64) + 160$	10,400
	Linear$(160 \to 64)$	$(64 \times 160) + 64$	10,304
	$2 \times$ LayerNorm(64)	$2 \times (64 + 64)$	256
	Subtotal per layer		37,600
Encoder (6 layers)		$6 \times 37{,}600$	225,600
Classifier	LayerNorm(64)	$64 + 64$	128
	Linear$(64 \to 10)$	$(10 \times 64) + 10$	650
Total Trainable			**230,730**

2.6.3. Strengths and limitations

2.6.3.a Strengths

- **Global context from the first layer:** Self-attention lets every patch interact directly with every other patch, so the model can relate distant parts of an item (for example collar and hem) without needing many stacked layers.
- **Parallel computation:** All 50 tokens are processed at the same time, in contrast to the step-by-step processing of a recurrent network, which makes the model well suited to GPUs.

- **Content-dependent interactions:** The attention weights are computed from the input itself, so the model decides which patches are relevant for each image instead of using fixed connections.
- **Flexible, learned spatial relations:** Because few assumptions are built into the architecture, spatial structure is learned from data through the patch embeddings and the positional embedding.
- **Stable deep stacking:** Residual connections and layer normalization keep optimization stable when several layers are stacked, and the same block can be scaled by changing the depth or width.
- **Low attention cost for small inputs:** With patches, the sequence contains only 50 tokens, so each attention matrix is only 50×50.

2.6.3.b Limitations

- **Weak inductive bias:** The model has no built-in locality or translation equivariance, so it must learn from data what a convolution gets for free. It may therefore need more data or augmentation than a CNN to reach similar performance on a dataset of this size.
- **Coarse tokenization:** Each 4×4 patch is flattened and projected linearly, so fine pixel-level detail inside a patch is compressed, and shifting an item by a number of pixels that is not a multiple of the patch size changes the tokens.
- **Fixed positional layout:** The learned positional embedding is tied to a 7×7 patch grid, so the model cannot directly handle images of a different size.
- **Quadratic attention cost:** The size of the attention matrix grows with the square of the number of tokens, so smaller patches (more tokens) increase computation and memory quickly.
- **Sensitivity to optimization:** Transformers are often sensitive to the learning rate, warm-up, depth, and number of heads, which must be chosen by trial and error.
- **Limited interpretability:** Attention maps give hints about which patches are used, but they are not a reliable explanation of the final prediction.

2.6.4. Self-implemented components and library dependencies

Self-implemented: Patch extraction, patch projection, `[CLS]` token, positional embedding, and classifier head (`PatchTransformer`, no pretrained backbone).

Library components:

The encoder layers (`nn.TransformerEncoderLayer`, `nn.TransformerEncoder`, which provide multi-head attention, feed-forward network, residual connections, and layer normalization), `nn.LayerNorm`, `nn.Linear`, dropout, and GELU from PyTorch.

2.7. Shared loss and evaluation metrics

2.7.1. Loss function

All five models use multiclass cross-entropy loss. For a batch of B images, each model outputs a logit matrix $\mathbf{Z} \in \mathbb{R}^{B\times 10}$, where each row contains ten unnormalized class scores. For image i with ground-truth label y_i, the loss is

$$\ell_i = -\log\left(\frac{\exp(Z_{i,y_i})}{\sum_{c=0}^{9}\exp(Z_{i,c})}\right). \tag{28}$$

The batch loss is the mean over its samples:

$$\mathcal{L} = \frac{1}{B}\sum_{i=1}^{B}\ell_i. \tag{29}$$

2.7.2. Predictive evaluation metrics

Test top-1 accuracy is the primary metric for model comparison. Secondary metrics include Macro-Precision, Macro-Recall, Macro-F1, per-class one-vs-rest accuracy, precision, recall, and F1-score, together with test cross-entropy loss.

2.7.2.1 Notation and confusion counts.

Let $K = 10$ denote the number of classes and N the number of evaluated images. For each class c, a one-vs-rest interpretation treats class c as positive and all remaining classes as negative:

- TP_c (true positives): images whose true and predicted labels are both class c.
- FN_c (false negatives): images belonging to class c that are predicted as another class.
- FP_c (false positives): images belonging to another class that are predicted as class c.
- TN_c (true negatives): images whose true and predicted labels are both different from class c.

For each class, $TP_c + FN_c + FP_c + TN_c = N$. These counts are computed from predictions collected across the complete evaluated dataset.

2.7.2.2 Overall predictive metrics.

Top-1 Accuracy: For each image, the predicted label is the class with the highest output logit. Top-1 accuracy measures the proportion of images whose predicted labels match their ground-truth labels:

$$\text{Top-1 Accuracy} = \frac{\sum_{c=0}^{K-1} TP_c}{N}. \tag{30}$$

Each correctly classified image contributes to exactly one class's true-positive count. This metric summarizes overall classification correctness, but does not show which classes account for the errors.

Macro-Precision: Macro-Precision is the unweighted average of precision across the K classes:

$$\text{Macro-Precision} = \frac{1}{K}\sum_{c=0}^{K-1}\frac{TP_c}{TP_c + FP_c}. \tag{31}$$

Each class contributes equally, regardless of how frequently it is predicted. The metric summarizes how reliable the model's class assignments are, with false positives reducing precision.

Macro-Recall: Macro-Recall is the unweighted average of recall across the K classes:

$$\text{Macro-Recall} = \frac{1}{K}\sum_{c=0}^{K-1}\frac{TP_c}{TP_c + FN_c}. \tag{32}$$

It summarizes how successfully the model recognizes images from each class, giving every class equal weight. False negatives reduce recall.

The Fashion-MNIST test set contains 1,000 images per class. Each class-wise recall therefore has the same denominator, making Macro-Recall equal to overall top-1 accuracy in this evaluation. This equality does not generally hold for datasets with unequal class support.

Macro-F1: Macro-F1 is the unweighted average of the individual class F1-scores:

$$\text{Macro-F1} = \frac{1}{K}\sum_{c=0}^{K-1}\frac{2TP_c}{2TP_c + FP_c + FN_c}. \tag{33}$$

Each class F1-score balances precision and recall, so the metric accounts for both false positives and false negatives. Macro-F1 is computed by averaging class-wise F1-scores, not by taking the harmonic mean of Macro-Precision and Macro-Recall.

2.7.2.3 Per-class predictive metrics.

Per-class metrics are reported separately for each garment category to identify strengths and weaknesses hidden by the overall averages. The support of class c is $TP_c + FN_c$, which equals 1,000 for every class in the test set.

Per-class one-vs-rest accuracy: This metric measures the proportion of images correctly identified as belonging or not belonging to class c:

$$\text{Accuracy}_c^{\text{OvR}} = \frac{TP_c + TN_c}{TP_c + TN_c + FP_c + FN_c}. \tag{34}$$

It includes true negatives. An image can contribute to TN_c even if it is misclassified between two other classes.

For Fashion-MNIST, each class has 1,000 positive and 9,000 negative images. A classifier that never

predicts class c would therefore achieve 90% one-vs-rest accuracy for that class, despite recognizing none of its images. This metric should consequently be interpreted alongside precision, recall, and F1-score. Its average across classes is not the primary top-1 accuracy metric.

Per-class precision: Precision measures how many predictions assigned to class c are correct:

$$P_c = \frac{TP_c}{TP_c + FP_c}. \tag{35}$$

For example, Shirt precision measures the proportion of images predicted as Shirt that actually belong to Shirt. Low precision indicates that images from other classes are frequently assigned to this category.

Per-class recall: Recall measures how many images belonging to class c are correctly recognized:

$$R_c = \frac{TP_c}{TP_c + FN_c}. \tag{36}$$

For example, Shirt recall measures the proportion of actual Shirt images correctly predicted as Shirt. Low recall indicates that many images of this class are assigned to other categories.

Per-class F1-score: The F1-score is the harmonic mean of precision and recall:

$$F1_c = \frac{2P_cR_c}{P_c + R_c} = \frac{2TP_c}{2TP_c + FP_c + FN_c}. \tag{37}$$

It provides a combined measure of class-specific precision and recall. A high F1-score requires both reliable class assignments and successful recognition of images belonging to the class.

Undefined precision, recall, or F1 values are set to zero, consistent with `zero_division=0` in scikit-learn. All accuracy, precision, recall, and F1 values are multiplied by 100 when reported as percentages. Test cross-entropy loss is reported separately using the loss function defined previously.

2.8. Training setup

To ensure fairness, reproducibility, and rigorous comparative analysis, an identical and strictly standardized training protocol was enforced across all five models. Table 9 summarizes the exact hyperparameter configurations and experimental environment.

Table 9: Standardized Training and Optimization Hyperparameters

Hyperparameter / Component	Specification / Value
Optimizer	`torch.optim.Adam` (default $\beta_1 = 0.9$, $\beta_2 = 0.999$; no weight decay) with `CrossEntropyLoss`
Learning Rate	Initial lr $= 10^{-3}$, minimum learning rate of 10^{-6}
Batch Size	128 (Train) | 256 (Validation and Test)
Maximum Epochs	200 (bounded by early stopping)
Learning Rate Scheduler	`ReduceLROnPlateau` (`mode=min`, factor $= 0.5$, patience $= 3$, threshold $= 10^{-4}$, `threshold_mode=abs`), stepped on validation loss
Data Augmentation (train only)	random rotation up to 10 degrees, horizontal flipping with probability 0.5, and random cropping to 28-by-28 pixels with padding 2; no augmentation on validation or test
Regularization	**Linear:** None. **MLP:** `Dropout` (0.3). **CNN:** `BatchNorm2d`, `Dropout2d` with rates 0.25, 0.25, and 0.3; `Dropout` (0.3). **LSTM:** Inter-layer and head `Dropout` (0.3). **Transformer:** Pre-`LayerNorm`, `Dropout` (0.1).
Early Stopping Criterion	Validation loss; patience $= 8$ epochs
Checkpoint Criterion	Minimum validation loss (`best_val_loss`); best weights restored before test evaluation
Seeds	Split Seed: 42, Training Seeds: 42,43,44
Hardware & Environment	NVIDIA Tesla T4 GPU, Google Colab (Linux, Python 3.13.15, PyTorch 2.11.0+cu130, Torchvision 0.26.0+cu130, numpy_version 2.1.3)
Numerical Precision	Full single precision (FP32, `torch.float32`; no mixed precision used)
Training Time	Measured across the complete training loop, including batch loading and preprocessing, training augmentation, device transfer, forward and backward passes, optimizer updates, metric computation, and validation. Scheduler updates, logging, and best-state copies in memory are also included. Dataset preparation before training and checkpoint file writing after training are excluded.
Hyperparameter Selection Method	Matched capacity budget for mandatory models ($\sim$ 231K–243K trainable parameters; Linear baseline: 7,850 parameters) and manual tuning on the validation split

2.9. Experimental Design

Two experiments are conducted. The first compares the five model configurations, while the second examines the effect of disabling dropout in the four nonlinear models.

	Experiment 1: main comparison	**Experiment 2: dropout ablation**
Hypothesis	Nonlinear models are expected to outperform the linear classifier. The CNN is expected to achieve the highest accuracy because its local receptive fields and shared filters provide a spatial inductive bias suited to garment images. The MLP, row-based LSTM, and patch-based Transformer provide alternative feature-learning configurations.	We test whether the original dropout settings improve predictive performance compared with disabling dropout under unchanged model configurations and training policies.
Changed factors	Architecture and input representation: flattened pixels for Linear and MLP, two-dimensional feature maps for CNN, row sequences for LSTM, and patch tokens for Transformer. Depth, hidden dimensions, normalization, and dropout settings also differ across architectures.	All dropout probabilities are set to zero, including spatial, classifier, recurrent inter-layer, embedding, and attention dropout where applicable. Architecture and trainable parameter count remain unchanged within each pair.
Models	Linear, MLP, CNN, LSTM, and Transformer.	Baseline and NoDropout variants of MLP, CNN, LSTM, and Transformer. Linear is excluded because it contains no dropout.
Fixed factors	The same stratified split of 48,000 training and 12,000 validation images, together with the official 10,000-image test set. Normalization statistics are computed exclusively from the training split. Both experiments share the augmentation settings, cross-entropy loss, Adam optimizer with initial learning rate 10^{-3}, training/evaluation batch sizes of 128/256, learning-rate scheduling policy, maximum budget of 200 epochs, and early-stopping patience of eight. Actual learning-rate histories and training durations may differ across runs.	
Seeds	Training seeds 42, 43, and 44, with a fixed data-split seed of 42. In the ablation, Baseline and NoDropout results are paired within the same model and training seed.	
Checkpoint rule	Each model, variant, and seed independently selects the checkpoint with the lowest validation cross-entropy loss. The selected weights are restored before test evaluation.	

Table 10: Hypotheses and experimental settings for the main model comparison and dropout ablation.

	Experiment 1: main comparison	Experiment 2: dropout ablation
Metrics	Primary: test top-1 accuracy. Secondary: Macro-Precision, Macro-Recall, Macro-F1, per-class one-vs-rest accuracy, precision, recall, F1-score, and test cross-entropy loss. Results are summarized as mean $\pm$ sample standard deviation across three seeds. Parameter counts and training and inference costs are reported separately. Ablation differences are computed as NoDropout minus Baseline within each seed, then summarized across the three paired runs.	
Conclusion criteria	Hypotheses are assessed primarily using mean test accuracy across seeds. Secondary metrics establish the consistency and scope of improvements.	The effect of removing dropout is assessed using paired differences across seeds. Positive accuracy and Macro-F1 differences, together with negative test-loss differences, favor NoDropout. The signs and variation of individual seed-level differences are examined to assess consistency. The opposite pattern favors Baseline, while mixed results qualify the conclusion.

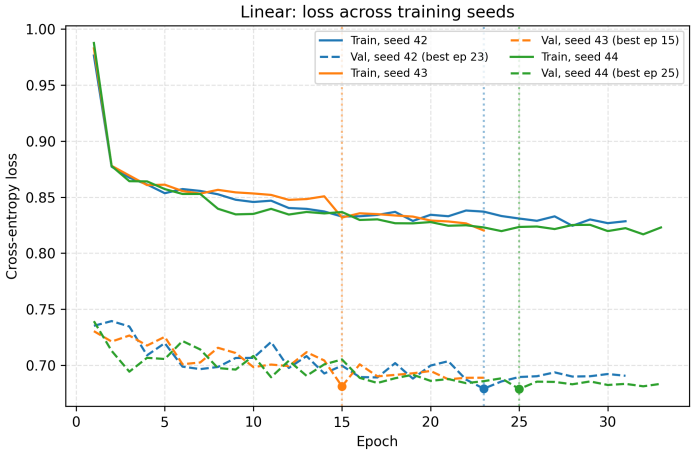
Table 11: Evaluation metrics and conclusion criteria for the main model comparison and dropout ablation.

3. Implementation Results

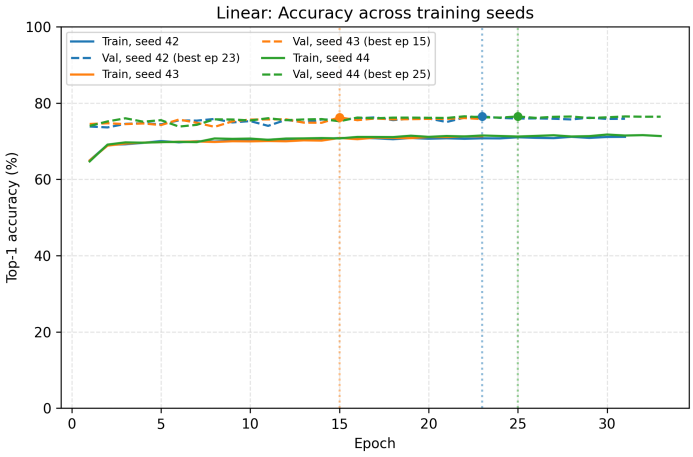
3.1. Training behavior

The following considerations apply to the learning curves of all five models. Training metrics are aggregated from batch predictions made throughout each epoch, during which model weights are updated after every batch. Validation metrics are computed afterward using the fixed weights obtained at the end of that epoch. Moreover, training uses augmented images, whereas validation uses unaugmented images. These differences can contribute to lower validation loss and higher validation accuracy than training ones. For models containing dropout or batch normalization, differences between training and evaluation modes may also affect the observed gap.

3.1.1. Baseline (Linear / softmax classifier)



(a) Training and validation loss curve



(b) Training and validation accuracy curve

Figure 7: Learning curves of the linear classifier for training seeds 42, 43, and 44.

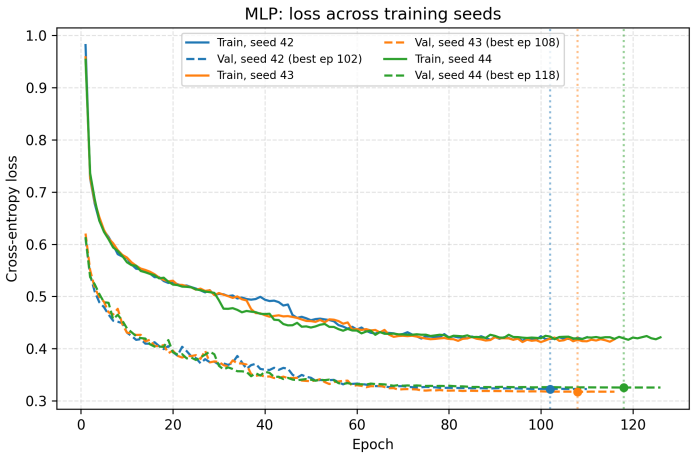
Figure 7 shows a similar learning pattern across all three training seeds. The linear classifier improves rapidly during the initial epochs, followed by slower progress and an eventual plateau. Around the selected checkpoints, training loss is approximately 0.82–0.84, while validation loss is close to 0.68. Training accuracy is approximately 71%, compared with approximately 76% for validation accuracy. For seed 43, the selected checkpoint records a training loss of 0.8318, a validation loss of 0.6813, and training and validation accuracy of 70.85% and 76.24%, respectively. Despite differences in their trajectories, the three runs reach similar validation performance.

For seeds 42, 43, and 44, the selected checkpoints occur at epochs 23, 15, and 25, respectively. Each checkpoint is selected by minimum validation loss, regardless of whether it also achieves the highest validation accuracy. Early stopping terminates the corresponding runs at epochs 31, 23, and 33, after eight consecutive epochs without improvement in validation loss. The different stopping points illustrate variation in optimization duration across training seeds, even though the resulting validation performance is similar.

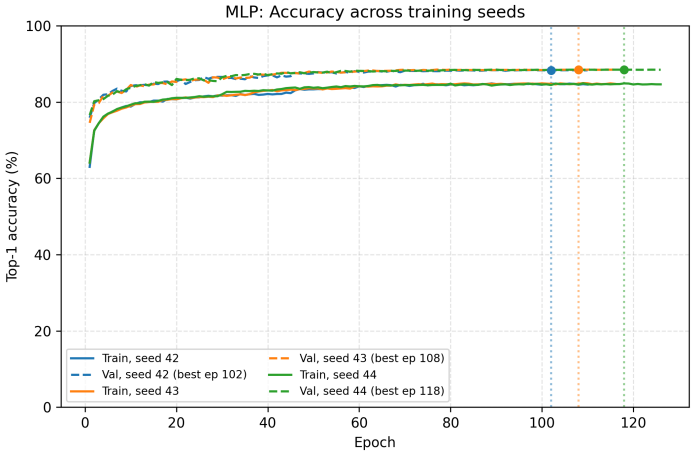
Total training times, including validation, are 574.5, 403.5, and 602.2 seconds for seeds 42, 43, and 44, respectively. Across the three runs, the mean total training time is 526.7 ± 107.6 seconds, while the mean training time per epoch is 18.11 ± 0.51 seconds. The variation in total training time mainly reflects the different numbers of executed epochs, with smaller differences in average epoch time.

After the selected checkpoints, validation loss rises slightly and fluctuates above its minimum, while training loss shows small further reductions. This behavior is compatible with mild overfitting, but validation performance does not progressively worsen throughout the remaining epochs. The curves therefore show no clear evidence of substantial late-stage overfitting. Overall, loss and accuracy approach a plateau, with limited additional validation gains. Together with the stronger performance of the nonlinear models, this behavior is consistent with the linear classifier's limited expressive capacity. However, the plateau alone does not conclusively establish underfitting.

3.1.2. Multilayer Perceptron



(a) Training and validation loss curve



(b) Training and validation accuracy curve

Figure 8: Learning curves of the MLP (Dashed vertical lines indicate the epochs selected by minimum validation loss)

Figure 8 shows a similar learning pattern across training seeds 42, 43, and 44. The MLP improves rapidly during the initial epochs, followed by more gradual gains and an eventual plateau. Around the selected checkpoints, training loss is approximately 0.42, while validation loss is approximately 0.32. Training accuracy approaches 85%, compared with approximately 88% for validation accuracy. For seed 43, the selected checkpoint records a training loss of 0.4171, a validation loss of 0.3173, and training and validation accuracy of 84.78% and 88.48%, respectively. Compared with the linear classifier, the MLP consistently reaches substantially lower losses and higher accuracy, consistent with the benefits of nonlinear feature learning.

The curves show further improvements following learning-rate reductions, although subsequent gains become progressively smaller. For example, seed 43 first reduces the learning rate at the end of epoch

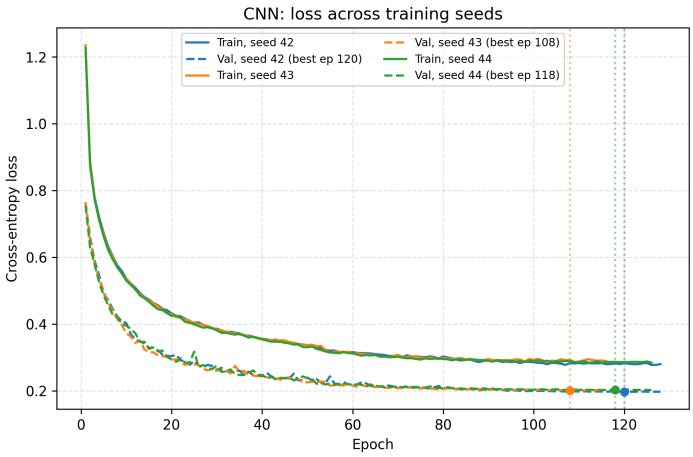
36, followed by a noticeable decrease in both training and validation loss.

For seeds 42, 43, and 44, the selected checkpoints occur at epochs 102, 108, and 118, respectively. Each run selects its checkpoint independently according to minimum validation loss, which need not coincide with maximum validation accuracy. For example, seed 43 records validation accuracy of 88.48% at its selected checkpoint, whereas validation accuracy subsequently reaches 88.51% at epoch 111 without improving validation loss. Early stopping terminates the corresponding runs at epochs 110, 116, and 126, after eight consecutive epochs without improvement in validation loss. Despite these different stopping points, the runs reach similar validation performance.

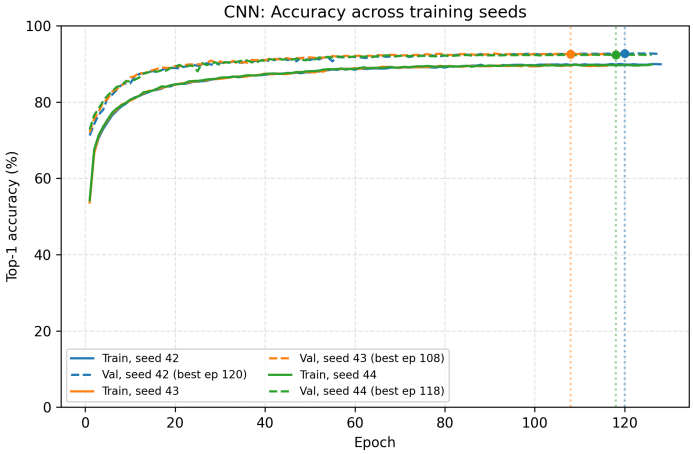
Total training times, including validation, are 2,065.6, 2,010.7, and 2,349.5 seconds for seeds 42, 43, and 44, respectively. Across the three runs, the mean total training time is $2{,}141.9 \pm 181.8$ seconds, while the mean training time per epoch is 18.25 ± 0.80 seconds. Differences in total training time reflect both the numbers of executed epochs and variation in average epoch time. In particular, seed 43 executes more epochs than seed 42 but finishes in less time because its average epoch time is shorter.

Toward the end of training, both training and validation performance largely stabilize, with only small fluctuations in loss and accuracy. Validation loss does not show a pronounced or sustained increase, so the curves provide no clear evidence of substantial late-stage overfitting. Further training offers limited validation gains under the current configuration. This plateau alone does not establish underfitting. However, the stronger results of the CNN, LSTM, and Transformer show that other evaluated configurations achieve better performance on this task.

3.1.3. Convolutional Neural Network (CNN)



(a) Training and validation loss curve



(b) Training and validation accuracy curve

Figure 9: Learning curves of the CNN (Dashed vertical lines indicate the epochs selected by minimum validation loss)

Figure 9 shows similar learning patterns across training seeds 42, 43, and 44. Loss decreases rapidly during the initial epochs, followed by more gradual improvement and an eventual plateau. Around the selected checkpoints, training loss is approximately 0.28–0.29, while validation loss is close to 0.20. Training accuracy approaches 90%, compared with approximately 92.5–93% for validation accuracy. For seed 43, the selected checkpoint records a training loss of 0.2932, a validation loss of 0.2014, and training and validation accuracy of 89.56% and 92.52%, respectively. Despite its lower initial accuracy, the CNN eventually reaches higher training and validation accuracy than both the linear classifier and MLP across the three runs.

Although the individual trajectories differ, all three runs show similar overall progress and final performance. Learning-rate reductions are followed by further improvements, with progressively smaller gains

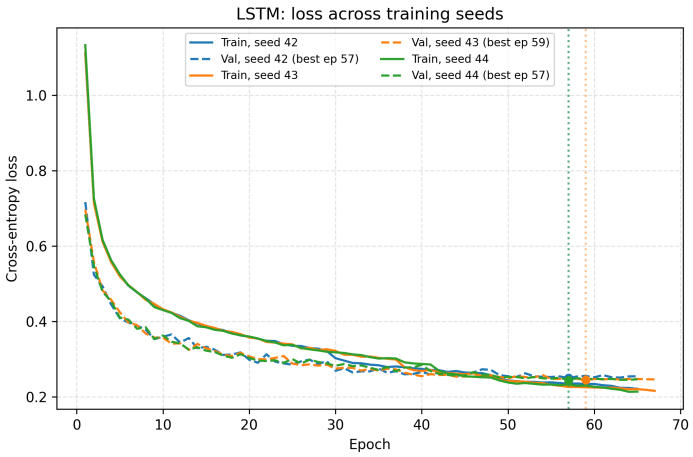
toward the end of training.

For seeds 42, 43, and 44, the selected checkpoints occur at epochs 120, 108, and 118, respectively. Each checkpoint is selected independently according to minimum validation loss rather than maximum validation accuracy. For example, seed 43 records validation accuracy of 92.52% at its selected checkpoint, although epoch 106 achieves higher validation accuracy of 92.65% with a higher validation loss. Early stopping terminates the corresponding runs at epochs 128, 116, and 126, after eight consecutive epochs without improvement in validation loss. The selected epochs differ, but checkpoint performance remains similar across the three seeds.

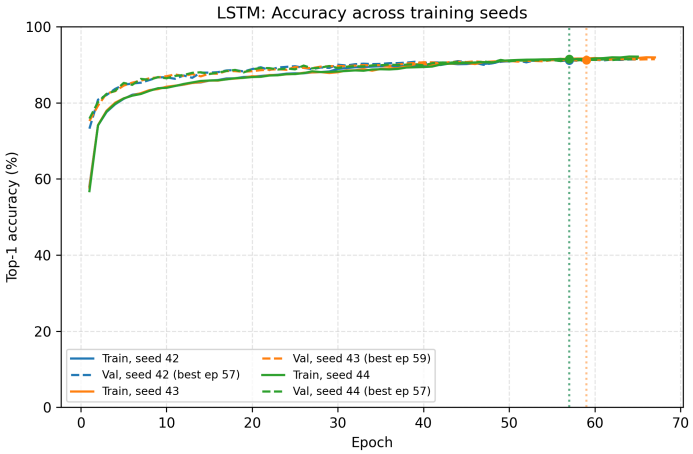
Total training times, including validation, are 2,512.7, 2,133.9, and 2,494.9 seconds for seeds 42, 43, and 44, respectively. Across the three runs, the mean total training time is $2{,}380.5 \pm 213.7$ seconds, while the mean training time per epoch is 19.28 ± 0.77 seconds. The differences in total training time reflect both the numbers of executed epochs and variation in average epoch time.

Toward the end of training, training and validation performance largely stabilize. After checkpoint selection, validation loss fluctuates slightly above its minimum without a pronounced or sustained deterioration in validation accuracy. These small changes do not provide clear evidence of substantial late-stage overfitting. Further training therefore produces limited validation gains under the current configuration. This plateau alone does not establish underfitting or demonstrate that further improvements are impossible.

3.1.4. Long Short Term Memory (LSTM)



(a) Training and validation loss curve



(b) Training and validation accuracy curve

Figure 10: Learning curves of the LSTM (Dashed vertical lines indicate the epochs selected by minimum validation loss)

Figure 10 shows rapid initial learning across training seeds 42, 43, and 44, followed by more gradual improvement. Around the selected checkpoints, training loss is approximately 0.22–0.24, while validation loss is approximately 0.24–0.25. Training and validation accuracy both approach 91–92%. For seed 43, the selected checkpoint records a training loss of 0.2249, a validation loss of 0.2446, and training and validation accuracy of 91.66% and 91.25%, respectively. The three runs follow similar overall trajectories and reach comparable validation performance.

Toward the end of training, the training and validation curves become more closely aligned than those of the linear classifier and MLP. Training loss eventually falls below validation loss in all three runs, while training accuracy becomes close to or slightly higher than validation accuracy.

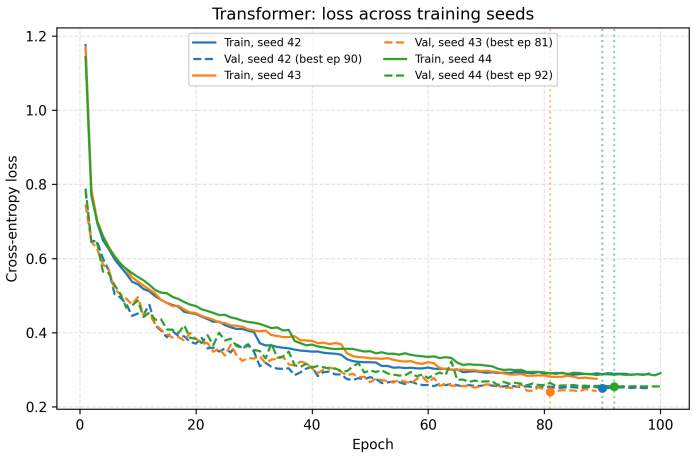
For seeds 42, 43, and 44, the selected checkpoints occur at epochs 57, 59, and 57, respectively. Each

checkpoint is selected independently by minimum validation loss, regardless of whether validation accuracy improves afterward. For example, seed 43 achieves validation accuracy of 91.25% at its selected checkpoint, whereas epoch 64 achieves 91.48% with a higher validation loss of 0.2469. Early stopping terminates the corresponding runs at epochs 65, 67, and 65, after eight consecutive epochs without improvement in validation loss. The selected and stopping epochs differ only slightly across seeds.

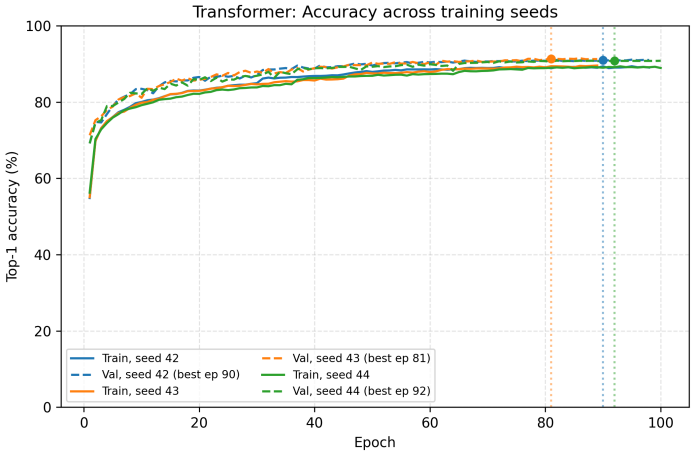
Total training times, including validation, are 1,284.7, 1,239.9, and 1,287.1 seconds for seeds 42, 43, and 44, respectively. Across the three runs, the mean total training time is $1{,}270.6 \pm 26.6$ seconds, while the mean training time per epoch is 19.36 ± 0.74 seconds. Although seed 43 executes two more epochs than the other runs, it finishes in less time because its average epoch time is shorter.

During the final epochs, validation performance largely stabilizes, while training loss continues to decrease. This developing separation is consistent with possible mild overfitting, but validation performance does not show a pronounced or sustained deterioration. For example, after checkpoint selection, seed 43's validation loss fluctuates between 0.2457 and 0.2471 while training loss decreases from 0.2249 to 0.2156. Further training therefore provides limited validation gains under the current configuration. Neither the small gap between the curves nor their stabilization alone is sufficient to establish underfitting.

3.1.5. Transformer



(a) Training and validation loss curve



(b) Training and validation accuracy curve

Figure 11: Learning curves of the Transformer (Dashed vertical lines indicate the epochs selected by minimum validation loss)

Figure 11 shows rapid initial improvement across training seeds 42, 43, and 44, followed by slower gains and an eventual plateau. At the selected checkpoints, training loss ranges from 0.2812 to 0.2880, while validation loss ranges from 0.2406 to 0.2546. Training accuracy ranges from 89.12% to 89.39%, and validation accuracy ranges from 90.83% to 91.29%. Seed 43 achieves the lowest validation loss and highest validation accuracy at its selected checkpoint, while seeds 42 and 44 reach more similar checkpoint performance.

Validation loss exhibits noticeable fluctuations during the early and middle stages of training, while training loss follows a smoother downward trend. These fluctuations generally become smaller toward the end of each run.

For seeds 42, 43, and 44, the selected checkpoints occur at epochs 90, 81, and 92, respectively, with

minimum validation losses of 0.2504, 0.2406, and 0.2546. Their corresponding validation accuracies are 90.94%, 91.29%, and 90.83%. Each checkpoint is selected independently according to minimum validation loss rather than maximum validation accuracy. For example, seed 43 subsequently reaches validation accuracy of 91.47% at epoch 86, but its validation loss of 0.2413 is higher than the selected minimum. Early stopping terminates the corresponding runs at epochs 98, 89, and 100, after eight consecutive epochs without improvement in validation loss. Similar overall learning patterns therefore coexist with differences in checkpoint selection and validation performance across seeds.

Total training times, including validation, are 2,200.0, 2,052.4, and 2,296.7 seconds for seeds 42, 43, and 44, respectively. Across the three runs, the mean total training time is $2{,}183.0 \pm 123.0$ seconds, while the mean training time per epoch is 22.83 ± 0.33 seconds. The variation in total training time mainly reflects the different numbers of executed epochs, with smaller differences in average epoch time.

During the final epochs, training loss changes only slightly, while validation loss fluctuates close to its minimum and validation accuracy remains approximately stable. For seed 43, training loss decreases from 0.2812 at the selected checkpoint to 0.2757 at the final epoch, while validation loss fluctuates between 0.2413 and 0.2478 afterward. This small separation is compatible with possible mild overfitting, but there is no pronounced or sustained deterioration in validation performance across the three runs. The curves therefore provide no clear evidence of substantial late-stage overfitting. Further training offers limited validation gains under the current configuration, but this plateau alone does not establish underfitting.

3.2. Quantitative results

The results in Table 12 correspond to the model architectures described in the Methodology section and the training settings summarized in Table 9. Each model is trained independently from scratch using training seeds 42, 43, and 44. All runs share the same Fashion-MNIST partitions, constructed using a fixed split seed of 42, and the same preprocessing and augmentation settings. Each configuration uses Adam with an initial learning rate of 10^{-3}, a training batch size of 128, and a maximum budget of 200 epochs. The actual number of epochs varies with early stopping. For each run, test evaluation uses the checkpoint with the lowest validation cross-entropy loss and an evaluation batch size of 256. Results are reported as mean $\pm$ sample standard deviation across the three training seeds.

Model size and computational cost

Model	Parameters	Epochs	Total train time per run (s)	Train time per epoch (s)	Inference (ms/sample)
Linear	7,850	29.00 ± 5.29	526.7 ± 107.6	18.11 ± 0.51	0.1681 ± 0.0062
MLP	242,762	117.33 ± 8.08	$2{,}141.9 \pm 181.8$	18.25 ± 0.80	0.1688 ± 0.0125
CNN	231,626	123.33 ± 6.43	$2{,}380.5 \pm 213.7$	19.28 ± 0.77	0.1727 ± 0.0010
LSTM	235,146	65.67 ± 1.15	$1{,}270.6 \pm 26.6$	19.36 ± 0.74	0.1804 ± 0.0047
Transformer	230,730	95.67 ± 5.86	$2{,}183.0 \pm 123.0$	22.83 ± 0.33	0.1738 ± 0.0052

Predictive performance

Model	Top-1 Accuracy. (%)	Macro-Precision (%)	Macro-Recall (%)	Macro-F1 (%)	Test CE loss
Linear	74.77 ± 0.35	74.49 ± 0.14	74.77 ± 0.35	74.48 ± 0.12	0.7141 ± 0.0028
MLP	87.30 ± 0.11	87.15 ± 0.10	87.30 ± 0.11	87.18 ± 0.10	0.3486 ± 0.0047
CNN	$\mathbf{91.54} \pm 0.20$	$\mathbf{91.52} \pm 0.19$	$\mathbf{91.54} \pm 0.20$	$\mathbf{91.51} \pm 0.19$	$\mathbf{0.2301} \pm 0.0030$
LSTM	90.85 ± 0.10	90.79 ± 0.08	90.85 ± 0.10	90.80 ± 0.09	0.2654 ± 0.0032
Transformer	90.43 ± 0.22	90.43 ± 0.18	90.43 ± 0.22	90.37 ± 0.25	0.2643 ± 0.0094

Table 12: Predictive performance, model size, and computational cost of the five models, reported as mean $\pm$ sample standard deviation across training seeds 42, 43, and 44.

Table 12 shows that the CNN achieves the strongest mean predictive performance, with a top-1 accuracy of $91.54 \pm 0.20\%$, Macro-Precision of $91.52 \pm 0.19\%$, Macro-Recall of $91.54 \pm 0.20\%$, Macro-F1 of $91.51 \pm 0.19\%$, and the lowest test cross-entropy loss of 0.2301 ± 0.0030. The LSTM has the smallest accuracy standard deviation, at 0.10 %, indicating the least variation across the three evaluated training seeds.

Macro-Recall equals top-1 accuracy for every run because the test set contains 1,000 images per class. Each class-wise recall is the number of correct predictions for that class divided by 1,000. Averaging these values over ten classes therefore gives the total number of correct predictions divided by 10,000, which is exactly the overall accuracy. Consequently, their means and standard deviations across seeds are also equal.

All four nonlinear models achieve higher mean accuracy, macro-averaged precision, recall, and F1-score, together with lower mean test loss, than the linear baseline. The CNN exceeds the linear classifier, MLP, LSTM, and Transformer in mean accuracy by 16.77, 4.24, 0.69, and 1.11 %, respectively. The LSTM ranks second, exceeding the Transformer by 0.42 %. However, the Transformer has a slightly lower mean test loss than the LSTM (0.2643 versus 0.2654), showing that the accuracy and loss rankings are

not identical.

The nonlinear models have comparable parameter counts, ranging from 230,730 to 242,762, whereas the linear classifier contains only 7,850 parameters. Mean epoch times are 18.11 ± 0.51, 18.25 ± 0.80, 19.28 ± 0.77, and 19.36 ± 0.74 seconds for the linear classifier, MLP, CNN, and LSTM, respectively. The Transformer requires the most time per epoch, at 22.83 ± 0.33 seconds. These measurements include data loading, augmentation, device transfer, forward and backward passes, optimizer updates, metric computation, and validation. They therefore describe the complete training pipeline rather than isolated model computation.

The linear classifier records the lowest mean inference time, at 0.1681 ± 0.0062 ms/sample, followed by the MLP at 0.1688 ± 0.0125, CNN at 0.1727 ± 0.0010, Transformer at 0.1738 ± 0.0052, and LSTM at 0.1804 ± 0.0047 ms/sample. These measurements include data loading, preprocessing, device transfer, and model forward passes with a batch size of 256.

The CNN provides the highest mean accuracy, while the LSTM reaches the second-highest accuracy with fewer training epochs. The LSTM stops after 65.67 ± 1.15 epochs, compared with 123.33 ± 6.43 for the CNN, while its mean accuracy is 0.69 % lower. The Transformer requires more time per epoch than both models and achieves lower mean accuracy under the evaluated setup.

Table 13: Per-class test metrics of the linear classifier, reported as mean ± sample standard deviation across three training seeds.

Class	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
T-shirt/top	94.81 ± 0.12	73.05 ± 1.15	76.20 ± 0.72	74.59 ± 0.32
Trouser	$\mathbf{97.98} \pm 0.02$	90.06 ± 1.25	$\mathbf{89.73} \pm 1.34$	$\mathbf{89.88} \pm 0.06$
Pullover	92.33 ± 0.29	62.59 ± 1.43	57.83 ± 2.84	60.10 ± 1.94
Dress	95.12 ± 0.07	73.30 ± 0.61	80.60 ± 0.95	76.77 ± 0.34
Coat	93.18 ± 0.19	65.46 ± 3.00	68.30 ± 5.97	66.65 ± 1.35
Sandal	96.29 ± 0.11	82.57 ± 0.88	79.77 ± 2.50	81.12 ± 0.92
Shirt	$\underline{88.71} \pm 0.52$	$\underline{42.83} \pm 1.41$	$\underline{37.43} \pm 7.16$	$\underline{39.64} \pm 3.65$
Sneaker	96.64 ± 0.12	81.33 ± 1.32	86.27 ± 1.80	83.71 ± 0.61
Bag	97.59 ± 0.09	$\mathbf{91.55} \pm 0.88$	83.63 ± 1.59	87.40 ± 0.58
Ankle boot	96.89 ± 0.12	82.19 ± 0.56	87.97 ± 0.68	84.98 ± 0.59

Table 14: Per-class test metrics of the MLP, reported as mean ± sample standard deviation across three training seeds.

Class	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
T-shirt/top	96.15 ± 0.06	79.18 ± 0.07	83.40 ± 0.79	81.23 ± 0.39
Trouser	$\mathbf{99.59} \pm 0.02$	$\mathbf{98.58} \pm 0.17$	$\mathbf{97.33} \pm 0.06$	$\mathbf{97.95} \pm 0.08$
Pullover	95.86 ± 0.07	78.96 ± 0.44	79.93 ± 0.15	79.44 ± 0.29
Dress	97.59 ± 0.09	86.44 ± 0.68	90.07 ± 0.25	88.21 ± 0.43
Coat	95.85 ± 0.09	78.58 ± 0.62	80.47 ± 0.49	79.51 ± 0.42
Sandal	99.09 ± 0.03	96.42 ± 0.34	94.37 ± 0.12	95.38 ± 0.15
Shirt	$\underline{93.40} \pm 0.06$	$\underline{69.65} \pm 0.28$	$\underline{60.20} \pm 0.46$	$\underline{64.58} \pm 0.36$
Sneaker	98.74 ± 0.05	92.42 ± 0.14	95.17 ± 0.49	93.78 ± 0.24
Bag	99.30 ± 0.01	96.26 ± 0.27	96.80 ± 0.26	96.53 ± 0.03
Ankle boot	99.03 ± 0.06	95.02 ± 0.62	95.27 ± 0.25	95.14 ± 0.27

Table 15: Per-class test metrics of the CNN, reported as mean ± sample standard deviation across three training seeds.

Class	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
T-shirt/top	97.25 ± 0.11	85.98 ± 0.66	86.63 ± 0.55	86.30 ± 0.53
Trouser	$\mathbf{99.79} \pm 0.02$	$\mathbf{99.29} \pm 0.00$	$\mathbf{98.57} \pm 0.21$	$\mathbf{98.93} \pm 0.11$
Pullover	97.79 ± 0.08	89.07 ± 0.42	88.80 ± 0.52	88.93 ± 0.38
Dress	98.28 ± 0.09	91.27 ± 0.63	91.57 ± 0.85	91.41 ± 0.45
Coat	97.32 ± 0.07	84.81 ± 0.64	89.13 ± 0.42	86.92 ± 0.27
Sandal	99.58 ± 0.03	98.84 ± 0.15	96.93 ± 0.35	97.88 ± 0.18
Shirt	$\underline{94.86} \pm 0.16$	$\underline{75.71} \pm 1.20$	$\underline{71.53} \pm 1.21$	$\underline{73.56} \pm 0.78$
Sneaker	99.15 ± 0.06	93.92 ± 0.32	97.87 ± 0.31	95.85 ± 0.27
Bag	99.72 ± 0.02	98.70 ± 0.10	98.53 ± 0.12	98.62 ± 0.08
Ankle boot	99.35 ± 0.04	97.59 ± 0.30	95.87 ± 0.31	96.72 ± 0.20

Table 16: Per-class test metrics of the LSTM, reported as mean ± sample standard deviation across three training seeds.

Class	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
T-shirt/top	96.98 ± 0.05	83.98 ± 0.44	86.30 ± 0.46	85.12 ± 0.22
Trouser	$\mathbf{99.78} \pm 0.02$	$\mathbf{99.16} \pm 0.06$	$\mathbf{98.60} \pm 0.17$	$\mathbf{98.88} \pm 0.11$
Pullover	97.12 ± 0.07	86.34 ± 0.92	84.60 ± 1.23	85.45 ± 0.38
Dress	98.25 ± 0.01	90.05 ± 0.82	92.80 ± 1.14	91.40 ± 0.13
Coat	97.02 ± 0.09	84.25 ± 1.34	86.43 ± 1.97	85.31 ± 0.54
Sandal	99.52 ± 0.05	97.89 ± 0.19	97.27 ± 0.57	97.58 ± 0.23
Shirt	$\underline{94.88} \pm 0.03$	$\underline{76.25} \pm 0.61$	$\underline{70.90} \pm 1.23$	$\underline{73.47} \pm 0.40$
Sneaker	99.16 ± 0.04	94.41 ± 0.20	97.33 ± 0.21	95.85 ± 0.20
Bag	99.64 ± 0.05	97.91 ± 0.44	98.47 ± 0.23	98.19 ± 0.23
Ankle boot	99.35 ± 0.02	97.69 ± 0.41	95.80 ± 0.26	96.74 ± 0.10

Table 17: Per-class test metrics of the Transformer, reported as mean ± sample standard deviation across three training seeds.

Class	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
T-shirt/top	97.04 ± 0.08	84.19 ± 0.87	86.73 ± 1.05	85.44 ± 0.37
Trouser	$\mathbf{99.70} \pm 0.04$	98.60 ± 0.39	$\mathbf{98.37} \pm 0.06$	$\mathbf{98.48} \pm 0.17$
Pullover	97.03 ± 0.29	82.32 ± 2.41	89.57 ± 0.55	85.78 ± 1.17
Dress	98.09 ± 0.11	89.71 ± 1.26	91.37 ± 0.35	90.52 ± 0.48
Coat	96.87 ± 0.12	84.67 ± 0.51	83.93 ± 0.75	84.30 ± 0.63
Sandal	99.40 ± 0.06	$\mathbf{98.93} \pm 0.42$	95.07 ± 1.01	96.96 ± 0.35
Shirt	$\underline{95.02} \pm 0.04$	$\underline{78.64} \pm 2.08$	$\underline{69.10} \pm 2.52$	$\underline{73.51} \pm 0.56$
Sneaker	98.94 ± 0.10	92.55 ± 0.94	97.23 ± 0.12	94.83 ± 0.44
Bag	99.62 ± 0.04	98.52 ± 0.46	97.63 ± 0.21	98.07 ± 0.17
Ankle boot	99.16 ± 0.03	96.20 ± 0.19	95.33 ± 0.38	95.76 ± 0.14

The high per-class accuracy values should be interpreted in the context of one-vs-rest evaluation. For each class, the test set contains 1,000 positive samples and 9,000 negative samples, so the large number of true negatives contributes heavily to accuracy. A classifier that never predicts a particular class would already achieve 90% one-vs-rest accuracy for that class, despite having zero recall.

Class-level analysis therefore emphasizes recall, which directly measures the proportion of images belonging to a class that are correctly recognized. Precision is examined alongside recall to assess how many predictions assigned to that class are correct, while F1-score summarizes the balance between precision and recall. The following comparisons use the mean metrics across the three training seeds.

Tables 13–17 identify Trouser as the most consistently easy class, with the highest mean one-vs-rest accuracy, recall, and F1-score within every model. Its mean recall ranges from 89.73% for the linear classifier to 98.60% for the LSTM. Bag is also among the easiest classes, particularly for the nonlinear models, whose mean Bag recall ranges from 96.80% to 98.53% and mean F1-score ranges from 96.53% to 98.62%. However, Trouser does not have the highest precision for every model: Bag ranks first for the linear classifier, while Sandal ranks first for the Transformer.

In contrast, Shirt has the lowest mean accuracy, precision, recall, and F1-score within every model, making it the most challenging class in this experiment. Its recall increases from $37.43 \pm 7.16\%$ for the linear classifier to $71.53 \pm 1.21\%$ for the CNN, an improvement of 34.10 % in the mean. Its F1-score likewise increases from $39.64 \pm 3.65\%$ to $73.56 \pm 0.78\%$. Despite these substantial gains, Shirt remains the weakest class even for the CNN.

The CNN's overall advantage does not extend to every class and metric. For example, the LSTM achieves higher mean Dress recall than the CNN (92.80% versus 91.57%), while their Dress F1-scores are nearly equal (91.40% versus 91.41%). For Shirt, the Transformer achieves higher precision than the CNN (78.64% versus 75.71%), but lower recall (69.10% versus 71.53%). Their Shirt F1-scores are also close (73.51% versus 73.56%). These results show that models can differ in precision and recall even when their F1-scores are similar.

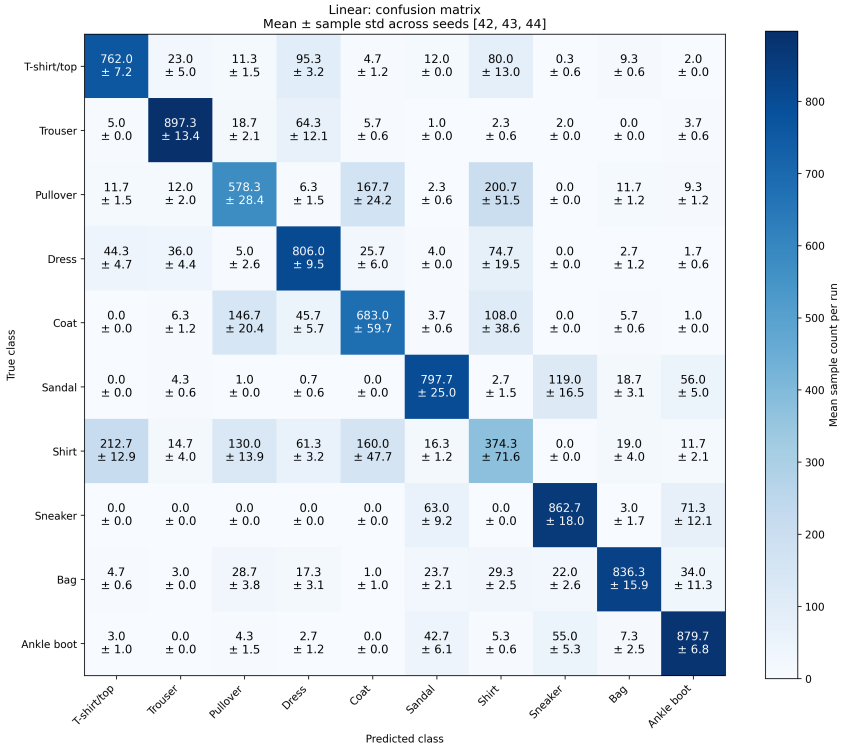


Figure 12: Test confusion matrix across three training seeds of the linear classifier.

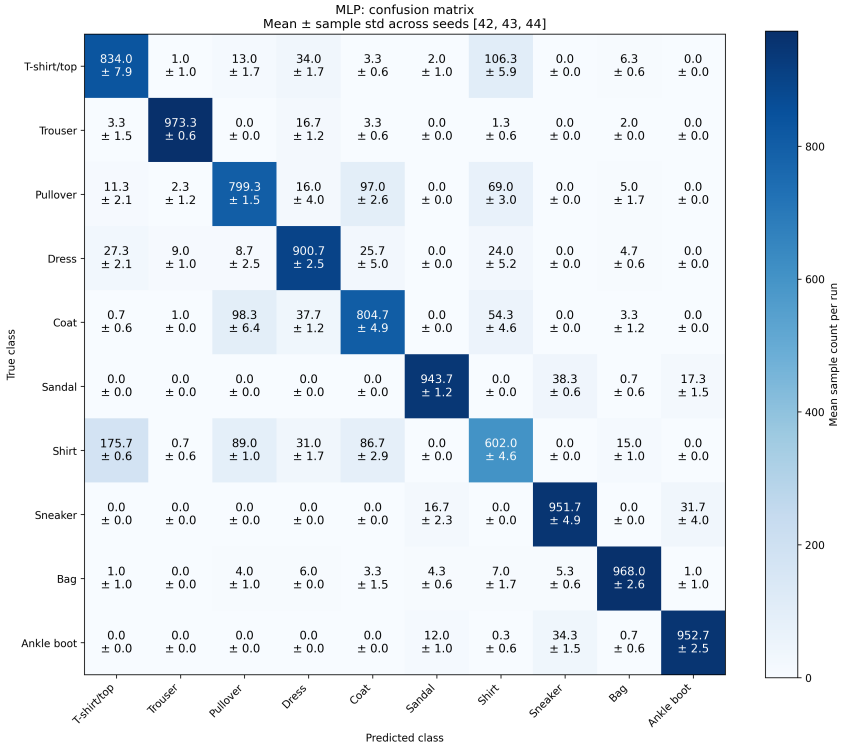


Figure 13: Test confusion matrix across three training seeds of the MLP.

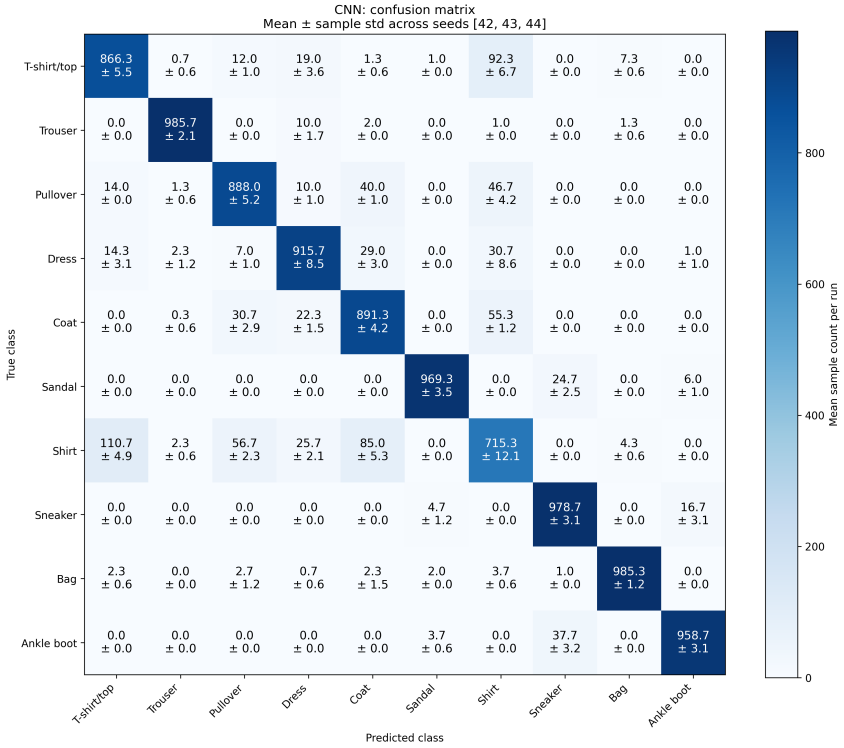


Figure 14: Test confusion matrix across three training seeds of the CNN.

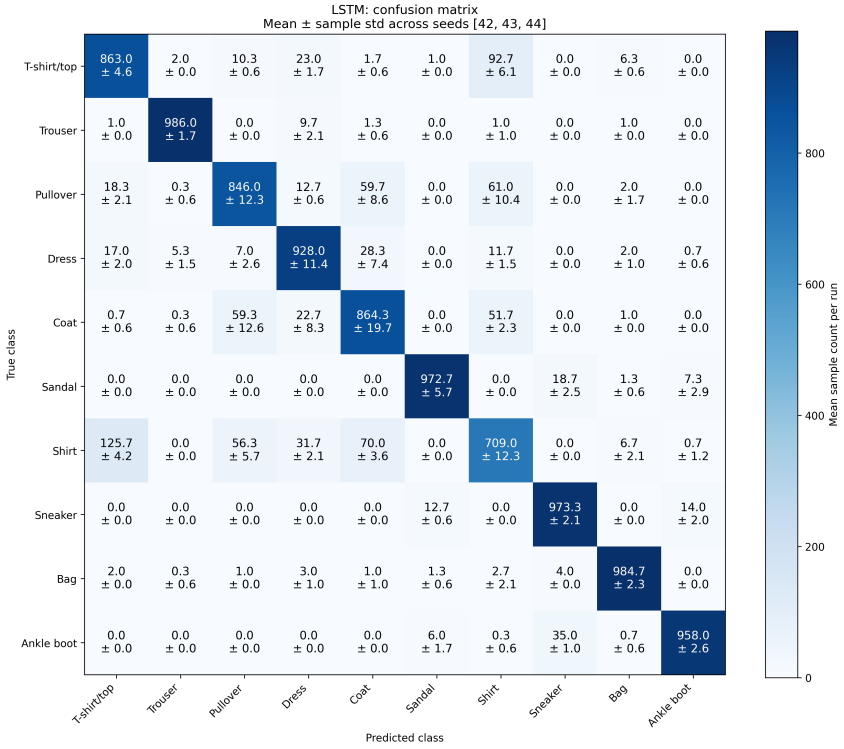


Figure 15: Test confusion matrix across three training seeds of the LSTM.

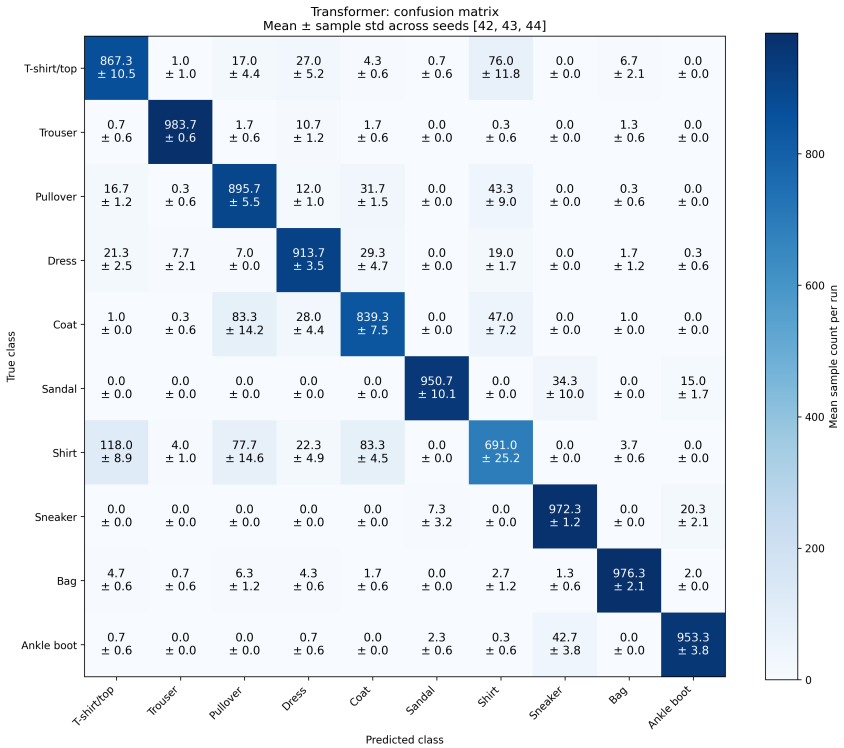


Figure 16: Test confusion matrix across three training seeds of the Transformer.

Figures 12–16 show the test confusion matrices of the five models. Rows represent true classes, columns represent predicted classes, and each cell reports the mean sample count ± sample standard deviation

across training seeds 42, 43, and 44. Each class contains 1,000 test samples per run, so mean diagonal counts can be compared directly across classes and models.

The nonlinear models achieve more correct predictions on average than the linear classifier in every class. However, all five models share a concentration of errors among upper-body garments, particularly Shirt, T-shirt/top, Pullover, and Coat. Shirt remains the most difficult class, with correct predictions increasing from 374.3 ± 71.6 for the linear classifier to 602.0 ± 4.6 for the MLP, 715.3 ± 12.1 for the CNN, 709.0 ± 12.3 for the LSTM, and 691.0 ± 25.2 for the Transformer. Its most frequent incorrect prediction is T-shirt/top for all five models. For this error, the mean count decreases from 212.7 for the linear classifier to 110.7 for the CNN.

The CNN substantially reduces confusion between Pullover and Coat. Mean Pullover-to-Coat errors decrease from 167.7 for the linear classifier and 97.0 for the MLP to 40.0 for the CNN. Errors in the reverse direction similarly decrease from 146.7 and 98.3 to 30.7. Nevertheless, some confusion remains asymmetric. For example, the Transformer predicts an average of 83.3 Coat images as Pullover, compared with 31.7 Pullover images predicted as Coat.

Trouser and Bag are generally recognized reliably, particularly by the nonlinear models. Although the CNN achieves the highest overall accuracy, its advantage does not extend to every class. The LSTM correctly recognizes more Dress images on average than the CNN (928.0 versus 915.7), along with slightly more Sandal images (972.7 versus 969.3). The Transformer correctly recognizes slightly more T-shirt/top images than the CNN (867.3 versus 866.3), as well as more Pullover images (895.7 versus 888.0).

3.3. Qualitative results

This section examines correctly classified images, difficult but correctly classified cases, and misclassified images for each model. All examples use the checkpoint selected by minimum validation loss for training seed 42. Each image is annotated with its ground-truth label and predicted label.

Difficult-but-correct candidates are selected from correct predictions with small differences between the two highest softmax scores, followed by visual inspection. These scores describe model confidence and do not, by themselves, establish image difficulty. The examples illustrate model behavior rather than estimate the frequency of each outcome.

3.3.1. Baseline (Linear / softmax classifier)

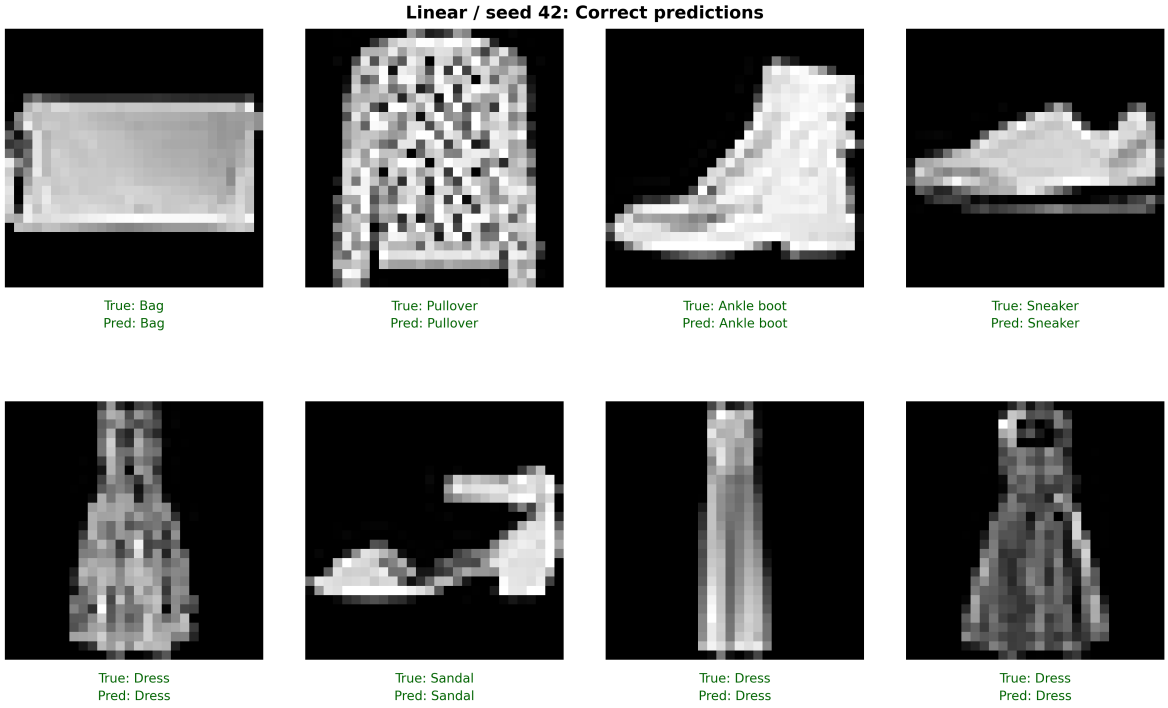


Figure 17: Correct predictions of the linear classifier for training seed 42.

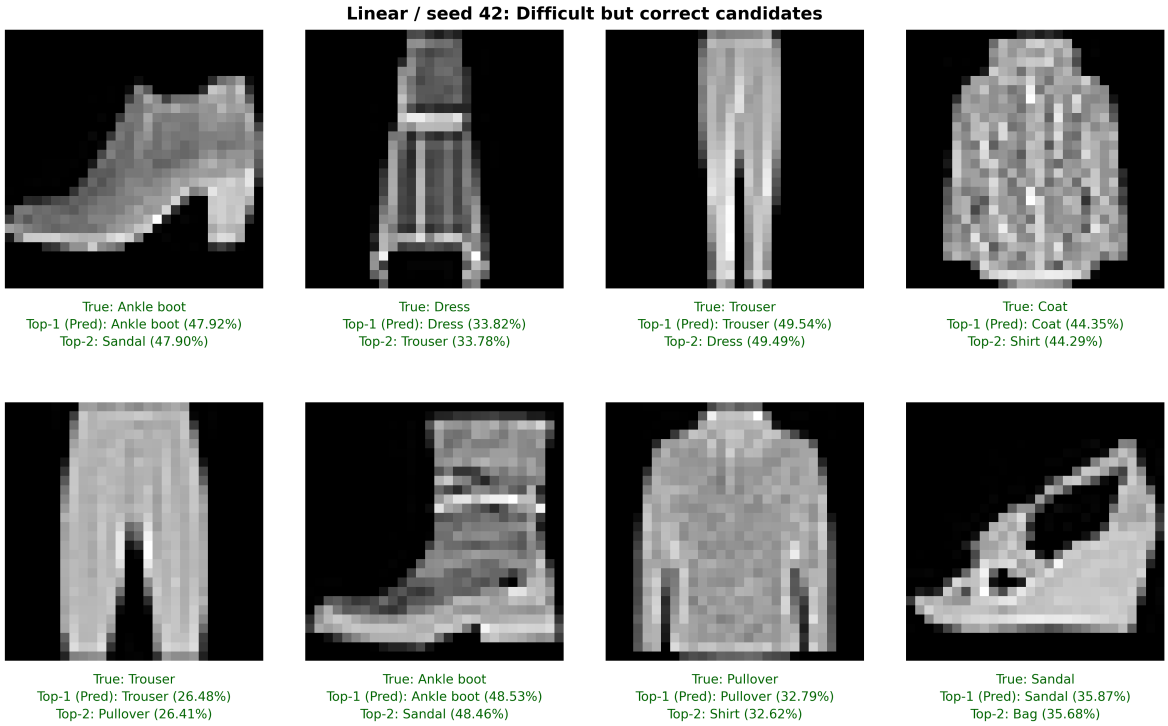


Figure 18: Difficult but correctly classified candidates of the linear classifier for training seed 42

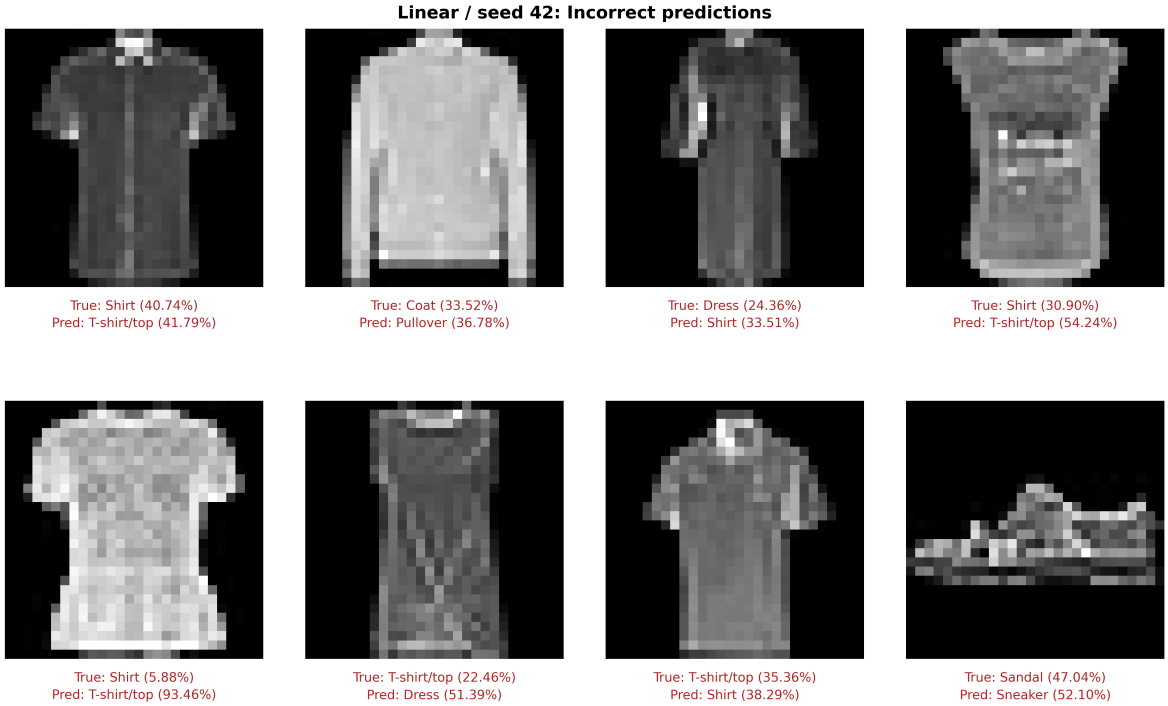


Figure 19: Incorrect predictions of the linear classifier for training seed 42

3.3.2. Multilayer Perceptron (MLP)

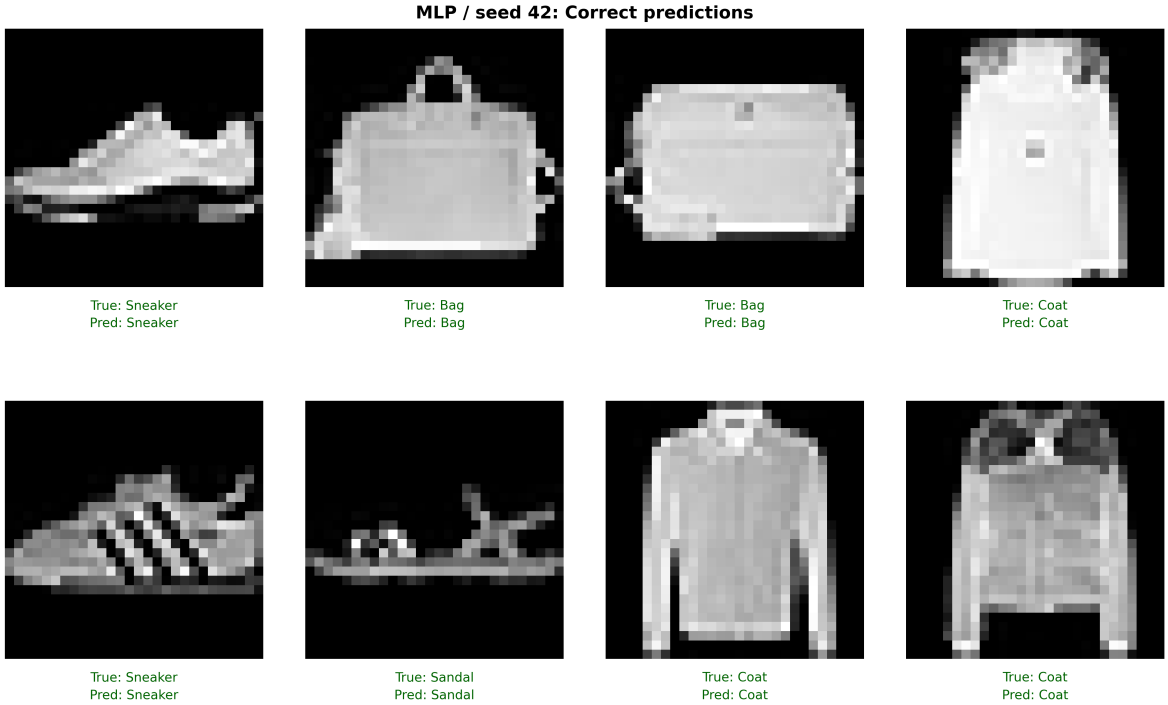


Figure 20: Correct predictions of the MLP for training seed 42.

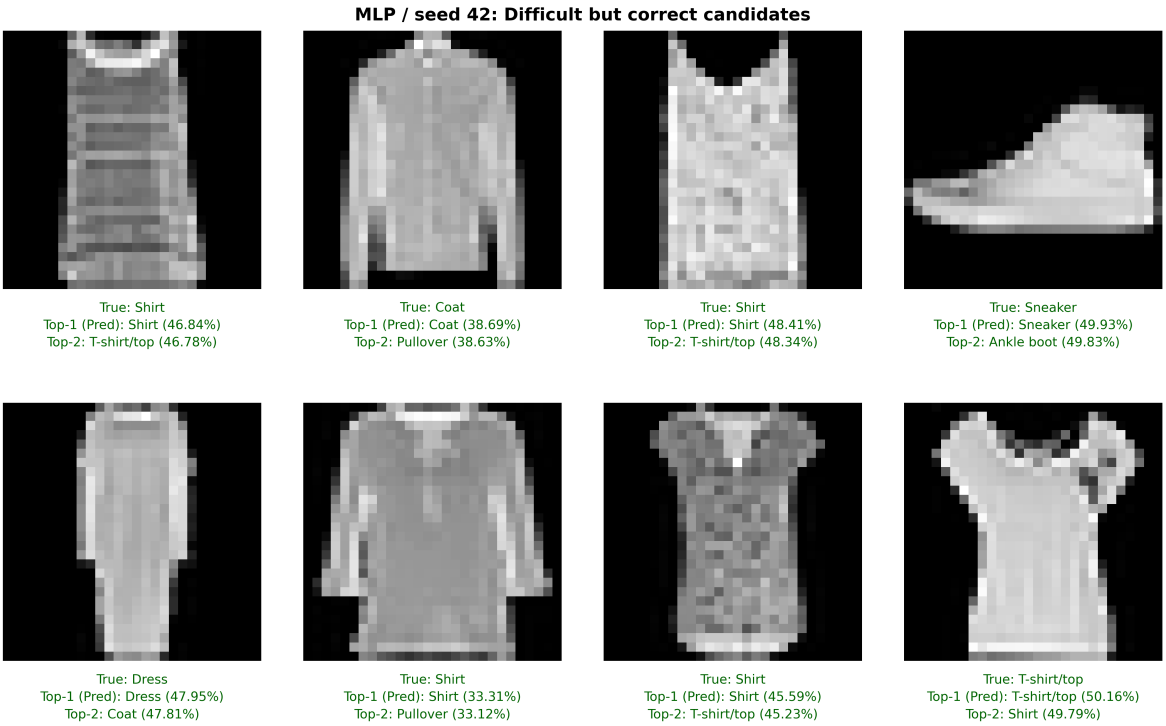


Figure 21: Difficult but correctly classified candidates of the MLP for training seed 42

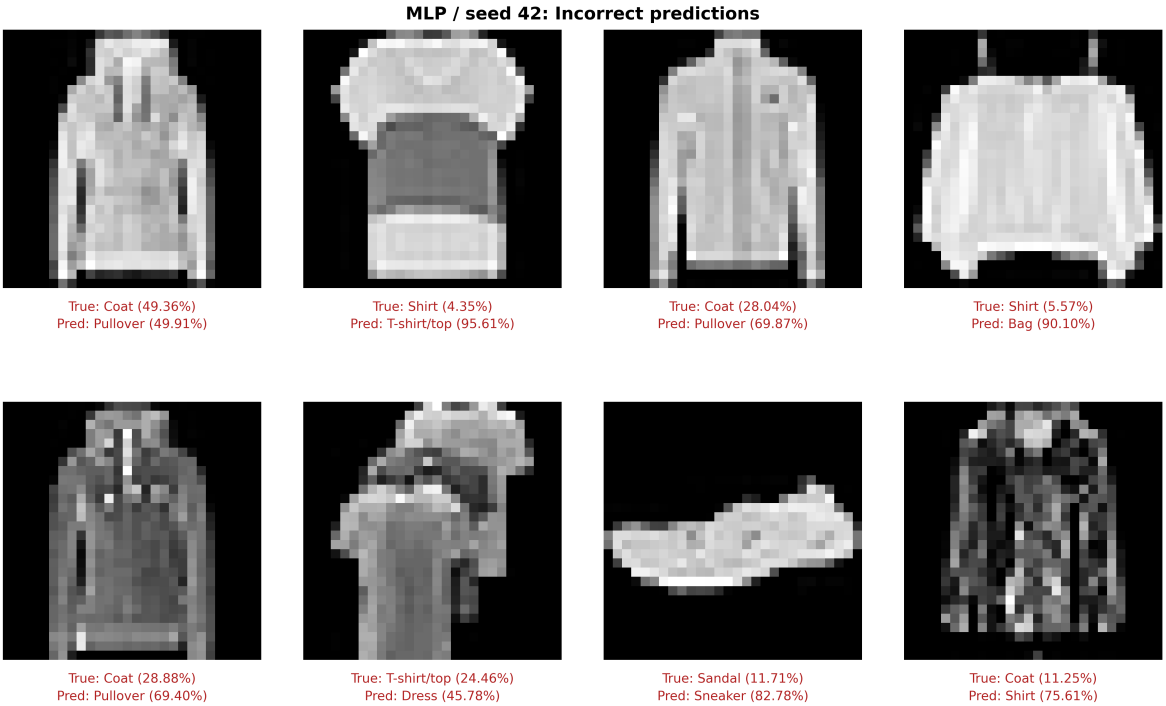


Figure 22: Incorrect predictions of the MLP for training seed 42

3.3.3. Convolutional Neural Network (CNN)

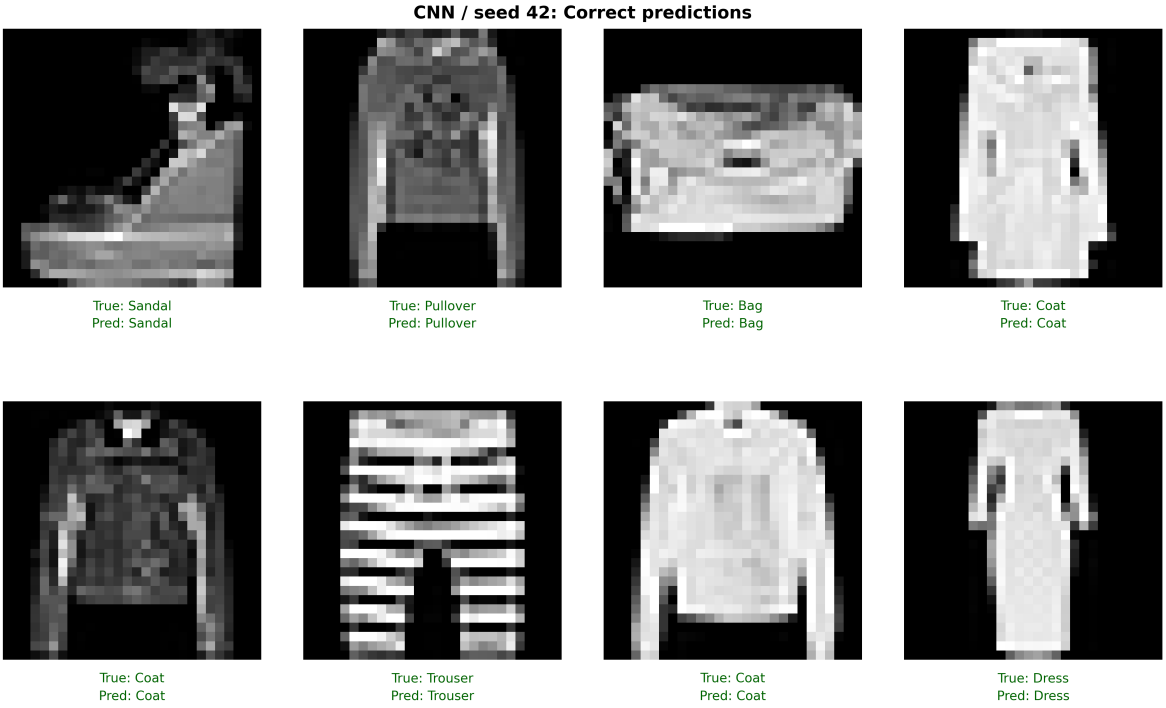


Figure 23: Correct predictions of the CNN for training seed 42.

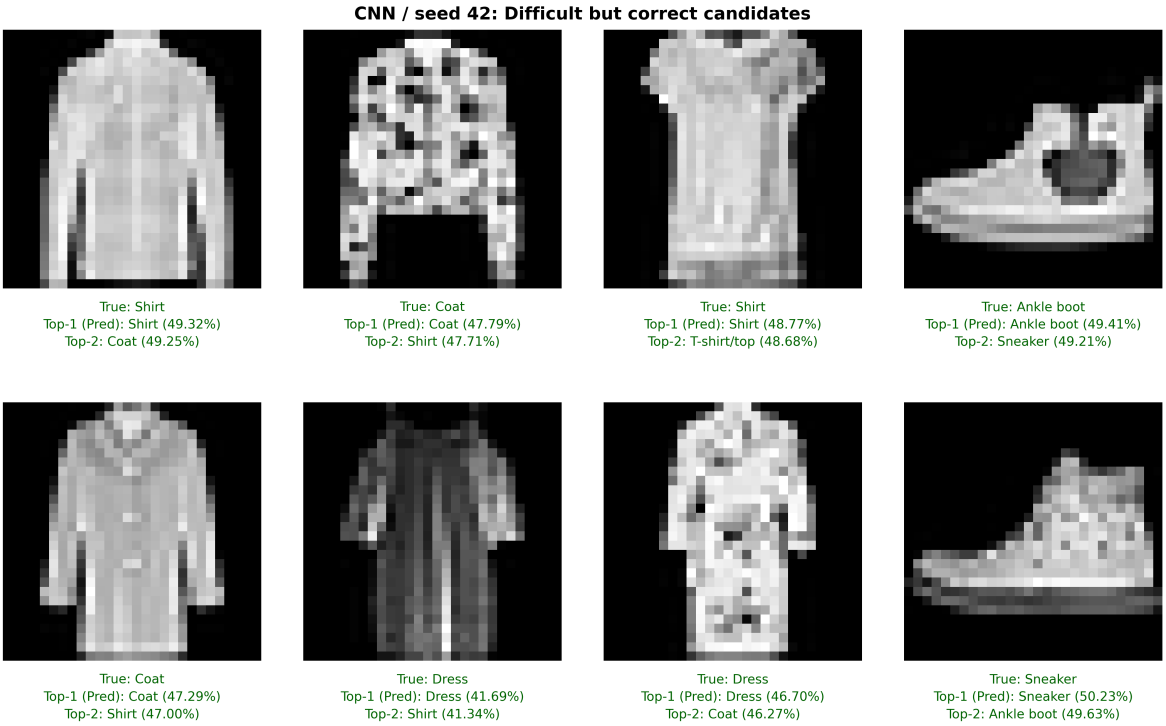


Figure 24: Difficult but correctly classified candidates of the CNN for training seed 42

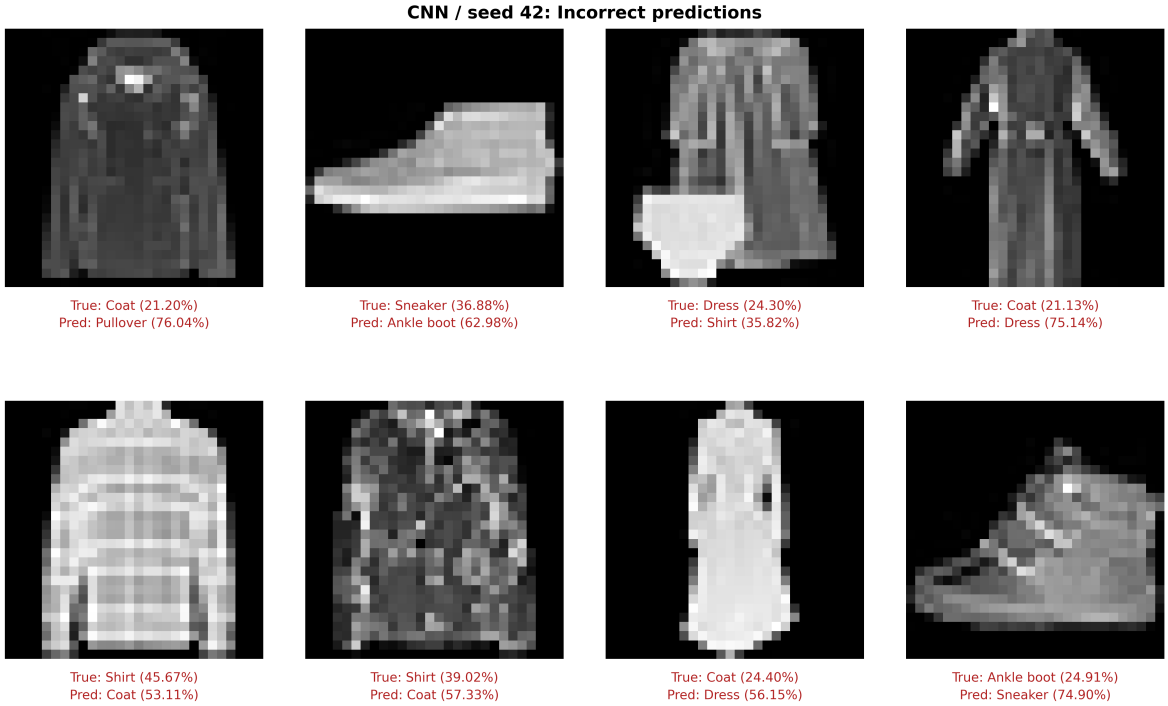


Figure 25: Incorrect predictions of the CNN for training seed 42

3.3.4. Long Short Term Memory (LSTM)

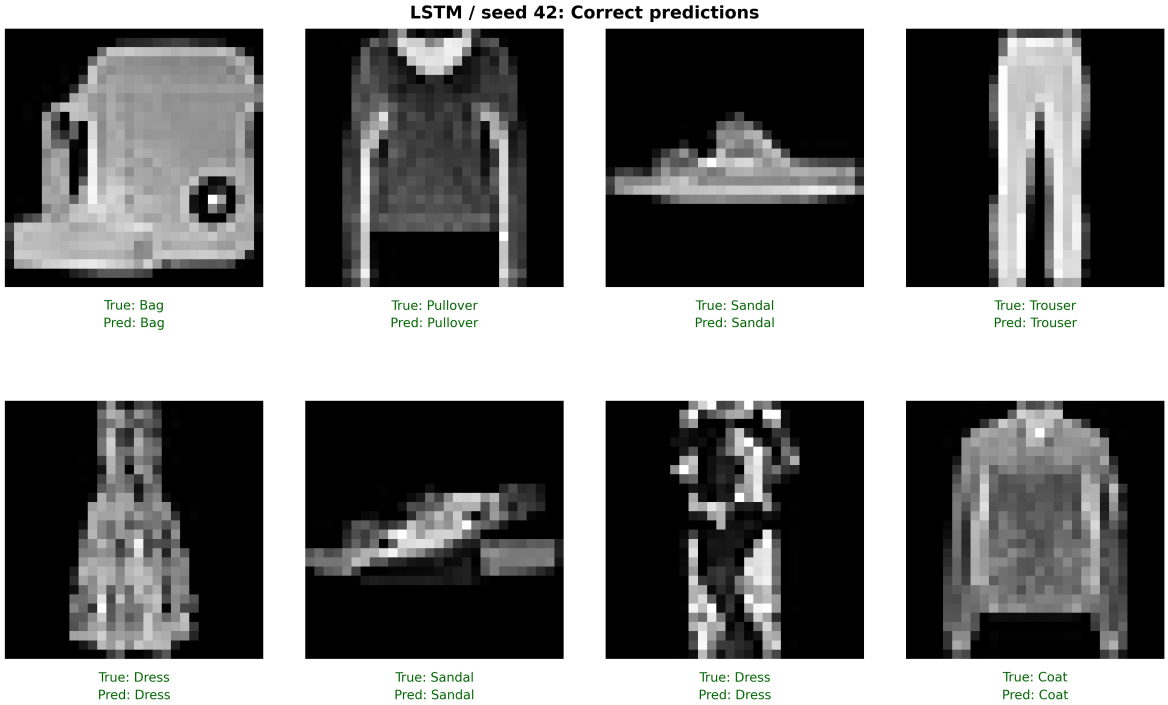


Figure 26: Correct predictions of the LSTM for training seed 42.

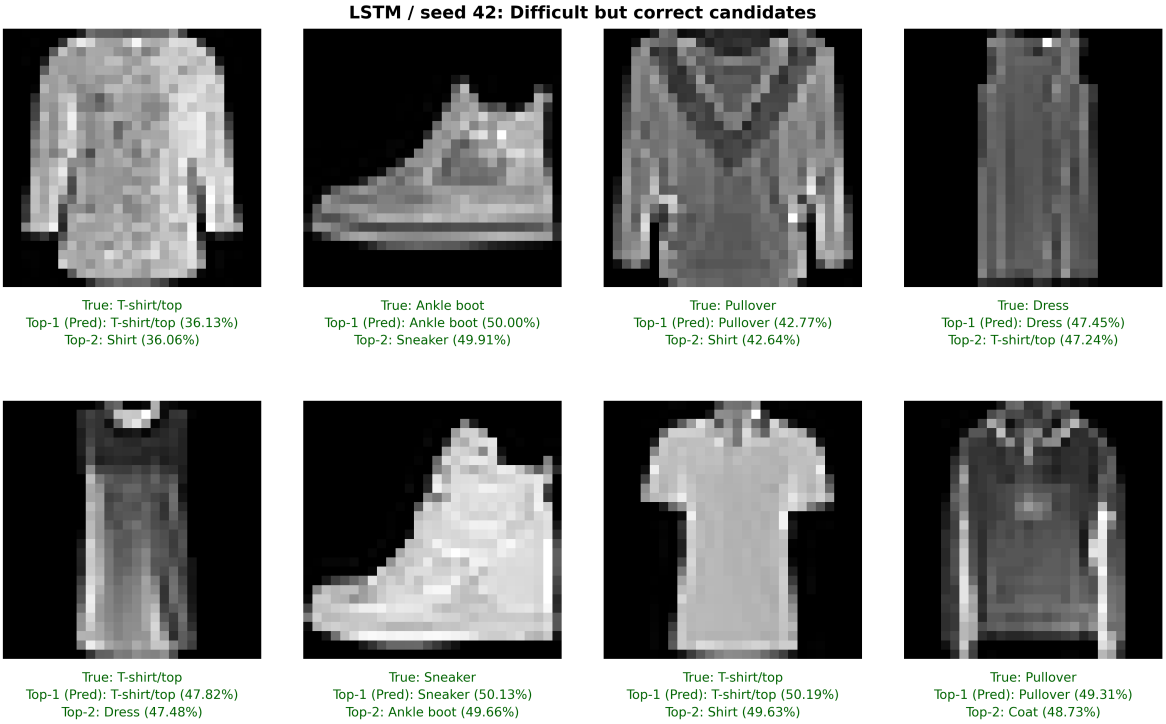


Figure 27: Difficult but correctly classified candidates of the LSTM for training seed 42

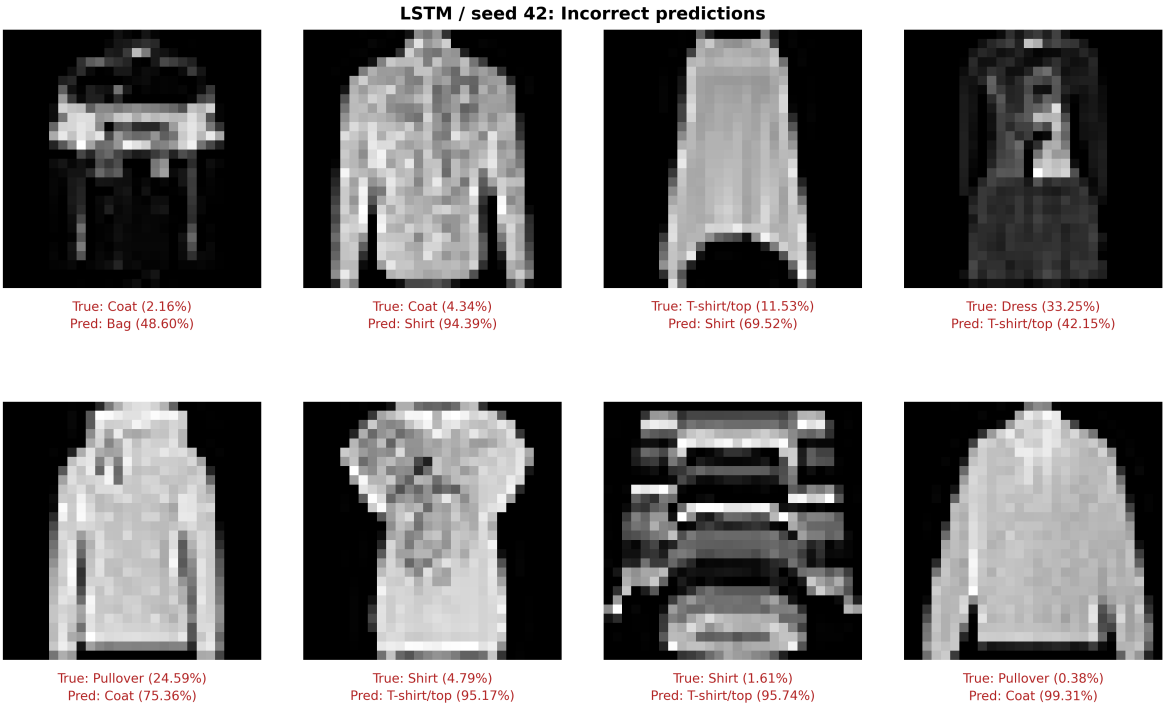


Figure 28: Incorrect predictions of the LSTM for training seed 42

3.3.5. *Transformer*

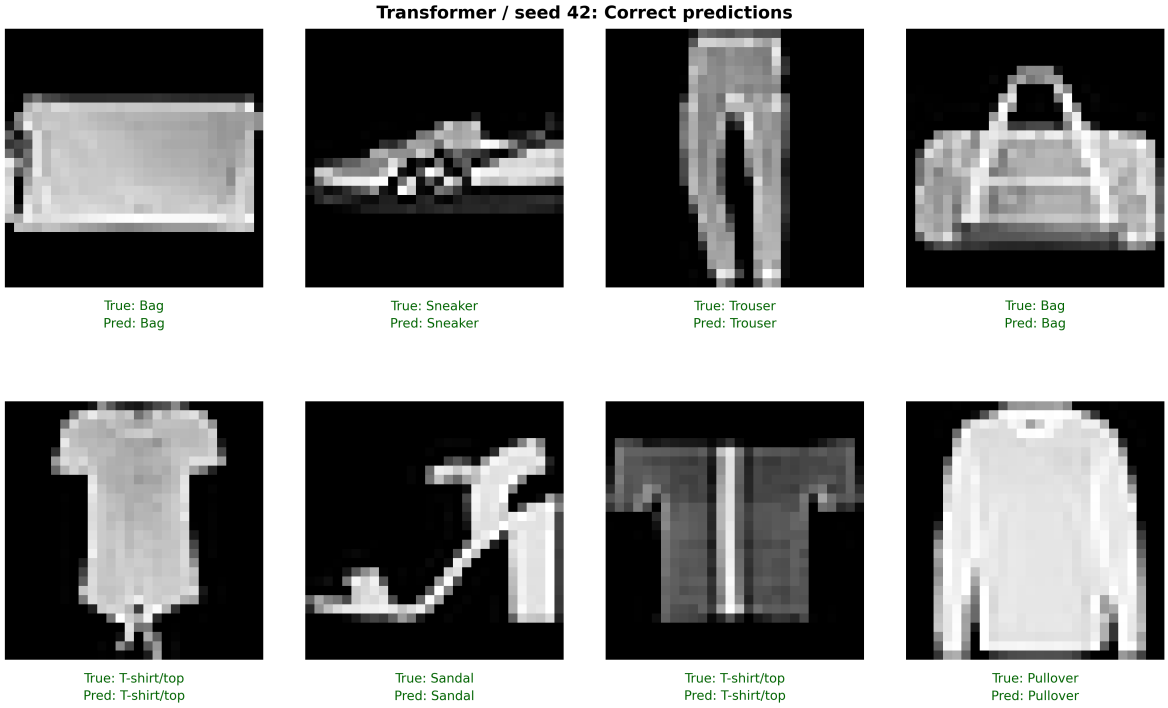


Figure 29: Correct predictions of the Transformer for training seed 42.

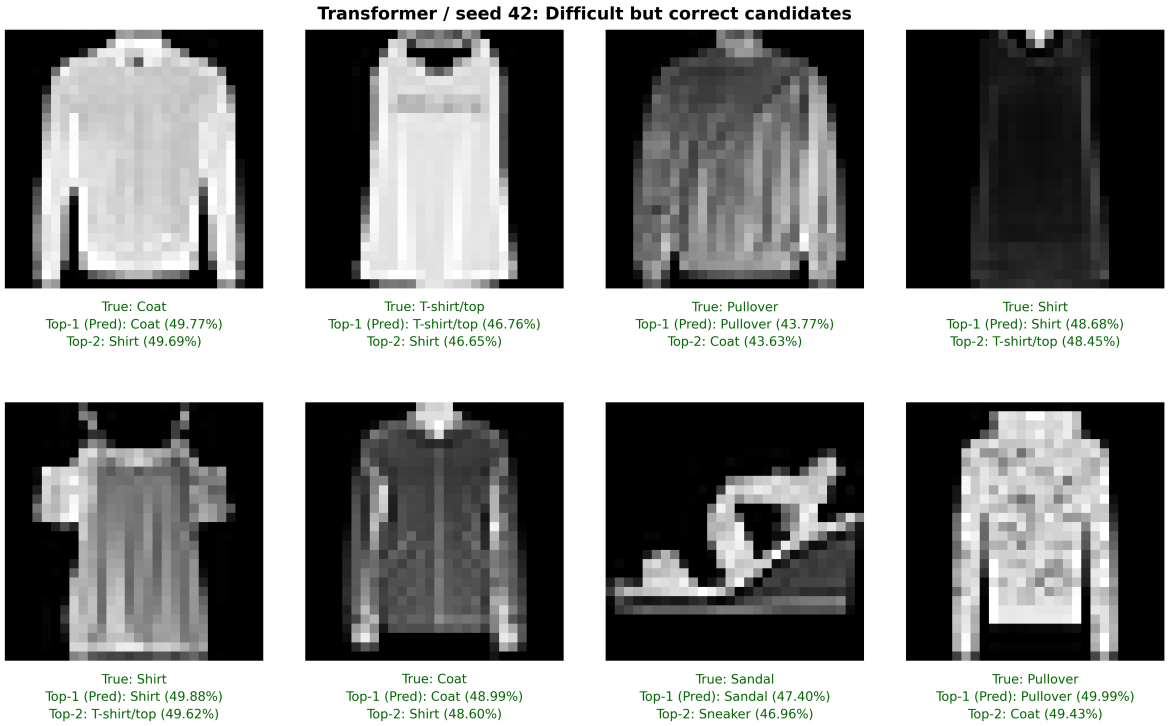


Figure 30: Difficult but correctly classified candidates of the Transformer for training seed 42

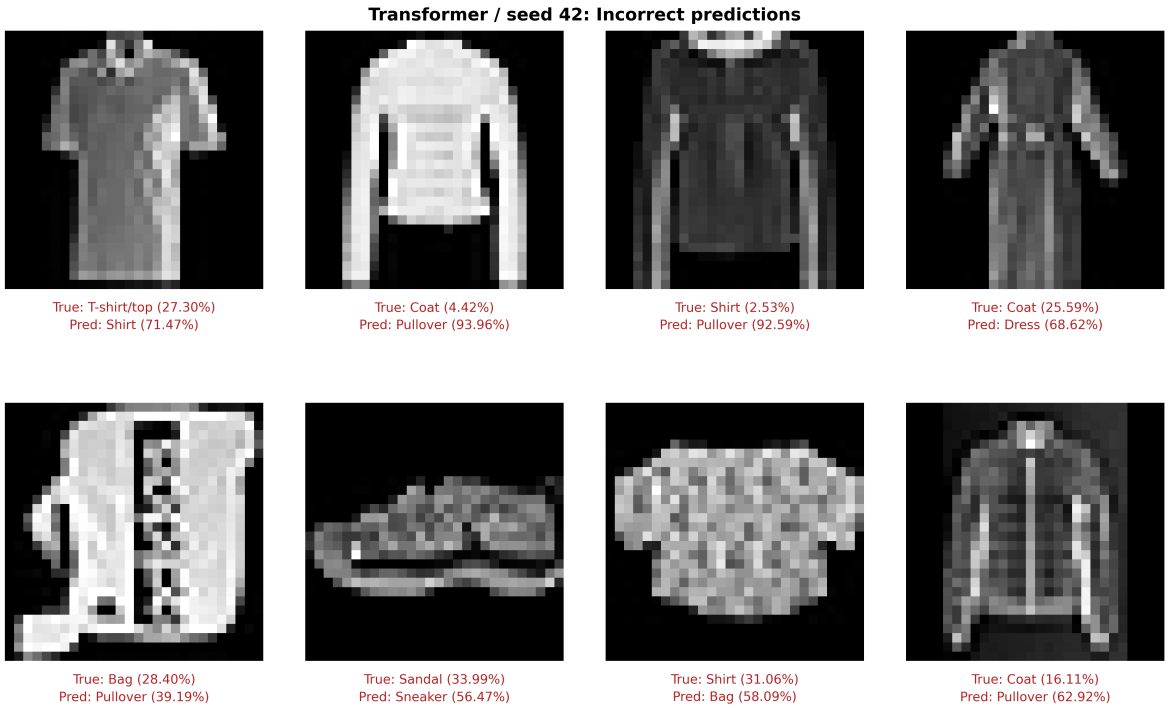


Figure 31: Incorrect predictions of the Transformer for training seed 42

3.4. Comparison and discussion

Taken together, the learning curves, test metrics, per-class results, confusion matrices, and qualitative examples show substantial improvements from the linear classifier to the nonlinear models.

3.4.1. Overall predictive performance

The CNN achieves the highest mean test top-1 accuracy of $91.54 \pm 0.20\%$, exceeding the linear classifier, MLP, LSTM, and Transformer by 16.77, 4.24, 0.69, and 1.11 %, respectively (Table 12). It also achieves the highest mean Macro-Precision and Macro-F1, together with the lowest mean test cross-entropy loss of 0.2301 ± 0.0030. Its overall advantage therefore extends beyond accuracy to the complementary predictive metrics. Macro-Recall equals accuracy because the test set is balanced, so these two metrics do not provide independent evidence of improvement.

The MLP improves mean accuracy over the linear classifier by 12.53 %. This result is consistent with the benefits of nonlinear feature learning, although the comparison also involves a substantial increase in parameter count. It therefore does not isolate the effect of nonlinearity from capacity. Among the nonlinear models, whose parameter counts are approximately comparable, the CNN still performs best despite having fewer parameters than the MLP.

The LSTM ranks second in mean accuracy, exceeding the Transformer by 0.42 %. However, the Trans-

former achieves a slightly lower mean test loss (0.2643 versus 0.2654). Thus, the accuracy ranking does not fully determine the loss ranking. The small loss difference should also be considered alongside the variation across training seeds.

3.4.2. Class-specific improvements and remaining errors

The CNN's improvements are particularly clear for difficult upper-body garment categories. Shirt recall increases from $37.43 \pm 7.16\%$ for the linear classifier to $71.53 \pm 1.21\%$ for the CNN, an improvement of 34.10 % in the mean. Its F1-score increases from $39.64 \pm 3.65\%$ to $73.56 \pm 0.78\%$. These gains show that the improvement in overall accuracy includes better recognition of a class that the linear classifier frequently misclassifies.

The confusion matrices provide more specific evidence of where these improvements occur. Mean Pullover-to-Coat errors decrease from 167.7 for the linear classifier to 40.0 for the CNN, while Coat-to-Pullover errors decrease from 146.7 to 30.7. Nevertheless, Shirt remains the weakest class within every model, and T-shirt/top is its most frequent incorrect prediction.

The qualitative examples suggest a plausible explanation for these recurring errors. Upper-body garments can share similar sleeves, body outlines, and visible patterns. At a resolution of 28×28 pixels, some distinguishing details are difficult to identify. These observations are consistent with the concentration of errors among Shirt, T-shirt/top, Pullover, and Coat, although they do not establish that the errors are unavoidable.

The CNN's overall advantage does not extend to every class-specific metric. For Dress, the LSTM achieves higher mean recall than the CNN (92.80% versus 91.57%), but lower precision (90.05% versus 91.27%). Their mean F1-scores are consequently almost equal (91.40% versus 91.41%). For Shirt, the Transformer achieves higher precision than the CNN (78.64% versus 75.71%), but lower recall (69.10% versus 71.53%). These examples illustrate why precision, recall, and F1-score should be examined together rather than treating overall accuracy as a complete description of model behavior.

3.4.3. Architecture and input representation

The CNN's performance is consistent with the suitability of its spatial inductive bias for garment images. Local receptive fields combine information from neighboring pixels, while shared convolutional filters reuse pattern detectors across image locations. These properties help the network learn garment contours and local structures more effectively than the flattened representations used by the linear classifier and MLP.

The LSTM and Transformer demonstrate that alternative input representations can also achieve strong performance. The LSTM processes images as sequences of rows, allowing its recurrent states to ac-

cumulate information across the image. However, this representation treats the two spatial directions differently and does not explicitly apply the same local two-dimensional filters as the CNN.

The Transformer processes patch tokens, with positional embeddings providing location information and self-attention learning relationships among patches. Its interactions are not restricted to neighboring patches, giving it a weaker local spatial inductive bias than the CNN. Under the evaluated dataset, parameter budget, and training settings, the CNN's built-in locality appears useful.

3.4.4. Loss, training behavior, and prediction confidence

All models use cross-entropy loss and the same general optimization and checkpoint-selection protocol. Their differences therefore cannot be attributed to different loss functions. However, the shared scheduling and early-stopping rules produce different learning-rate histories and training durations because they respond to each run's validation loss.

Cross-entropy depends on the probability assigned to the ground-truth class, whereas accuracy depends only on whether the highest-scoring class is correct. Consequently, minimum validation loss and maximum validation accuracy need not occur at the same epoch. For example, the Transformer with seed 43 selects epoch 81 with validation accuracy of 91.29%, although epoch 86 achieves 91.47% with a higher validation loss.

The qualitative results further distinguish classification correctness from confidence. In the MLP's difficult-but-correct examples, one Shirt image receives softmax scores of 46.84% for Shirt and 46.78% for T-shirt/top. Conversely, one misclassified Shirt image receives 90.10% for Bag and only 5.57% for its ground-truth class. Thus, correct predictions can have very small score margins, while incorrect predictions can strongly favor the wrong class. The CNN's lower mean test loss is favorable, but does not by itself establish that its softmax scores are calibrated.

The learning curves generally show rapid initial improvement followed by limited additional validation gains. The LSTM develops a small late-stage separation as training loss continues to decrease while validation performance stabilizes, which is consistent with possible mild overfitting. However, the curves do not show substantial, sustained validation deterioration.

Training augmentation and training-mode regularization also affect the observed gaps, so their sizes cannot directly measure overfitting. Similarly, a plateau alone does not establish underfitting. The linear classifier's lower performance is consistent with limited expressive capacity, but optimization and augmentation may also contribute.

3.4.5. Accuracy, execution time, and parameter trade-offs

The linear classifier contains only 7,850 parameters, making it the smallest configuration, but its mean accuracy is substantially lower than that of every nonlinear model. The nonlinear models contain approximately 231,000–243,000 parameters. Within this range, the CNN improves accuracy over the MLP by 4.24 % while using 11,136 fewer parameters. It therefore offers a more favorable balance of predictive performance and parameter count within the evaluated configurations.

Mean epoch times are 18.11 ± 0.51 seconds for the linear classifier, 18.25 ± 0.80 for the MLP, 19.28 ± 0.77 for the CNN, and 19.36 ± 0.74 for the LSTM. The CNN and LSTM therefore have similar measured epoch costs. However, the LSTM stops after 65.67 ± 1.15 epochs, compared with 123.33 ± 6.43 for the CNN, while sacrificing 0.69% in mean accuracy. It offers a practical alternative when shorter training under the shared early-stopping protocol is desirable.

The Transformer requires the most time per epoch, at 22.83 ± 0.33 seconds, while achieving lower mean accuracy than the CNN and LSTM. It therefore offers a less favorable accuracy–epoch-time balance under the evaluated setup. This observation does not establish that Transformer architectures are inherently less efficient.

The linear classifier records the lowest mean inference time, at 0.1681 ± 0.0062 ms/sample, followed closely by the MLP at 0.1688 ± 0.0125 ms/sample. The CNN, Transformer, and LSTM record 0.1727 ± 0.0010, 0.1738 ± 0.0052, and 0.1804 ± 0.0047 ms/sample, respectively. The small absolute differences and variation across runs limit how strongly this ordering can be interpreted.

Both training and inference measurements include shared pipeline operations. Training includes data loading, augmentation, device transfer, forward and backward passes, optimizer updates, metric computation, and validation. Inference includes data loading, preprocessing, device transfer, and forward passes. These measurements therefore do not isolate architecture-specific computation.

3.4.6. Hypothesis assessment and limitations

The results support the hypothesis that the evaluated nonlinear configurations outperform the linear classifier on the primary metric and the overall secondary predictive metrics. They also support the expectation that the CNN achieves the highest overall accuracy among the evaluated configurations. However, the class-specific exceptions show that this conclusion does not extend to every class and metric.

3.5. Error Analysis

3.5.1. Confusion between visually similar classes

Description and frequency: Recurring errors involve visually similar upper-body garments and footwear categories.

Among upper-body garments, Shirt is the most difficult class for every model. The CNN misclassifies 284.7 ± 12.1 of the 1,000 Shirt images, corresponding to $28.47 \pm 1.21\%$. Its most frequent incorrect predictions are T-shirt/top (110.7 ± 4.9 images) and Coat (85.0 ± 5.3 images). The mean Shirt-to-T-shirt/top error rate decreases from 21.27% for the linear classifier to 11.07% for the CNN.

Pullover and Coat are also confused in both directions. For the CNN, 40.0 ± 1.0 Pullover images are predicted as Coat ($4.00 \pm 0.10\%$), while 30.7 ± 2.9 Coat images are predicted as Pullover ($3.07 \pm 0.29\%$). The corresponding mean counts for the linear classifier are 167.7 and 146.7. The Transformer shows greater directional asymmetry, with 83.3 Coat images predicted as Pullover versus 31.7 Pullover images predicted as Coat on average.

Among footwear categories, the CNN predicts 16.7 ± 3.1 Sneaker images as Ankle boot ($1.67 \pm 0.31\%$) and 37.7 ± 3.2 Ankle boot images as Sneaker ($3.77 \pm 0.32\%$). Another 24.7 ± 2.5 Sandal images are predicted as Sneaker ($2.47 \pm 0.25\%$).

Examples: Figure 25 illustrates these patterns. A Shirt image is predicted as Coat with a softmax score of 53.11%, compared with 45.67% for Shirt. A Coat image receives 76.04% for Pullover and 21.20% for Coat. A Sneaker image receives 62.98% for Ankle boot and 36.88% for Sneaker.

Hypothesized cause: These errors may share a common difficulty: similar overall shapes combined with weak or poorly resolved distinguishing details. Upper-body garments can share sleeve shapes and body outlines, while footwear can share side profiles, soles, and upper contours. At 28×28 resolution, features such as necklines, collars, front openings, and ankle openings may be difficult to identify. Spatial downsampling could further reduce these details. These explanations remain hypotheses rather than established causes.

Improvement ideas: Controlled ablations could test reduced early downsampling or additional feature extraction before pooling to preserve local details. An augmentation ablation could also assess whether the current transformations help or hinder discrimination between similar classes. Any predictive gains should be assessed alongside parameter count and execution time.

For Shirt specifically, targeted sampling of difficult training examples or moderate class weighting could be evaluated. Since the dataset is balanced, this would address class difficulty rather than class imbalance. Evaluation should monitor class-specific precision, recall, and both directions of each confusion pair,

since reducing one error type may introduce others.

3.5.2. Incorrect predictions with high softmax scores

Description and frequency: Some incorrect predictions strongly favor the wrong class rather than showing a small difference between competing labels. This group overlaps with the class-based groups above. Its overall frequency has not been quantified, because the reported confusion matrices do not distinguish errors by confidence. Selected examples therefore establish that such failures occur, but not how common they are.

Examples: In Figure 22, a Shirt image is predicted as Bag with a softmax score of 90.10%, while Shirt receives only 5.57%. Another Shirt image is predicted as T-shirt/top with 95.61%, compared with 4.35% for Shirt. These cases contrast with errors where the predicted and ground-truth scores are nearly equal.

Hypothesized cause: A model may learn visual features that strongly support an incorrect category for a particular image. Cross-entropy training does not guarantee that a high softmax score corresponds to a correct prediction. However, these selected failures alone do not establish systematic overconfidence or identify the specific features responsible.

Improvement ideas: Temperature scaling fitted on the validation set could be evaluated on the test set. This method adjusts confidence scores without changing the predicted class

4. Ablation study

Dropout is intended to improve generalization by reducing co-adaptation between hidden units. However, its effectiveness may depend on the architecture, dropout probability, and other regularization mechanisms, including training augmentation. All dropout locations are disabled together. The ablation therefore assesses their combined effect rather than identifying which layer or dropout mechanism is responsible. For example, it does not distinguish the CNN's spatial dropout from its classifier dropout, or attention dropout from other dropout locations in the Transformer. Table 18 reports the NoDropout results as mean ± sample standard deviation across the three seeds.

Model	Top-1 Accuracy. (%)	Macro-Precision (%)	Macro-Recall (%)	Macro-F1 (%)	Test CE loss
MLP	88.33 ± 0.26	88.29 ± 0.26	88.33 ± 0.26	88.30 ± 0.26	0.3196 ± 0.0021
CNN	$\mathbf{92.26} \pm 0.15$	$\mathbf{92.24} \pm 0.13$	$\mathbf{92.26} \pm 0.15$	$\mathbf{92.23} \pm 0.15$	$\mathbf{0.2178} \pm 0.0052$
LSTM	90.63 ± 0.23	90.58 ± 0.23	90.63 ± 0.23	90.59 ± 0.23	0.2665 ± 0.0063
Transformer	91.65 ± 0.25	91.61 ± 0.27	91.65 ± 0.25	91.61 ± 0.26	0.2367 ± 0.0038

Table 18: Test predictive performance of the NoDropout variants, reported as mean ± sample standard deviation across training seeds 42, 43, and 44

Model	Δ Top-1 Accuracy. (%)	Δ Macro-Precision (%)	Δ Macro-Recall (%)	Δ Macro-F1 (%)	Δ Test CE loss
MLP	$+1.03 \pm 0.20$	$+1.14 \pm 0.19$	$+1.03 \pm 0.20$	$+1.12 \pm 0.19$	-0.0291 ± 0.0033
CNN	$+0.72 \pm 0.17$	$+0.72 \pm 0.15$	$+0.72 \pm 0.17$	$+0.72 \pm 0.16$	-0.0124 ± 0.0033
LSTM	-0.22 ± 0.33	-0.21 ± 0.31	-0.22 ± 0.33	-0.21 ± 0.32	$+0.0012 \pm 0.0095$
Transformer	$+1.21 \pm 0.16$	$+1.18 \pm 0.18$	$+1.21 \pm 0.16$	$+1.25 \pm 0.15$	-0.0276 ± 0.0066

Table 19: Paired predictive-performance differences between NoDropout and Baseline, reported as mean ± sample standard deviation across training seeds 42, 43, and 44.

Removing dropout improves accuracy, Macro-F1, and test loss for the MLP, CNN, and Transformer across all three evaluated seeds. The largest mean accuracy improvement occurs for the Transformer, at 1.21%. In contrast, the LSTM shows a small mean accuracy decrease of 0.22%. The results therefore indicate that the effect of dropout depends on the model configuration, rather than consistently benefiting or harming all evaluated architectures.

5. Limitations and conclusion

5.1. Limitations

5.1.1. Data limitations.

The study uses Fashion-MNIST images with a fixed resolution of 28×28 pixels and a single grayscale channel. Fine details such as collars, garment openings, and footwear structures can be difficult to distinguish at this resolution. The recurring confusion among similar upper-body garments is consistent with this limitation, although the experiments do not isolate the contribution of image resolution to these errors.

The balanced test set provides equal representation across classes, but does not evaluate performance under different class distributions. Furthermore, the dataset does not establish how the models would perform on photographs with different backgrounds, viewpoints, lighting conditions, or image acquisition procedures.

5.1.2. Method limitations.

Each architecture is represented by one specific configuration. Although the nonlinear models have approximately comparable parameter counts, they differ in depth, normalization, dropout, and input representation. The comparisons therefore do not isolate the contribution of convolution, recurrence, self-attention, or individual regularization components.

The shared optimization and augmentation settings provide a common experimental protocol, but may not be equally suitable for every architecture. Consequently, the results may not establish the best achievable performance of each model family.

5.2. Generalization and future work

The held-out test results provide evidence of generalization to unseen Fashion-MNIST images under the evaluated protocol. However, they do not establish robustness to distribution shifts or transfer to other image-classification datasets. The CNN's advantage should therefore be interpreted within the current dataset, configurations, and training conditions.

Future experiments could use additional training seeds and alternative training and validation partitions while retaining the official test set for final evaluation. Controlled ablations of dropout, normalization, augmentation, and spatial downsampling would help distinguish the contributions of individual design choices. A broader validation-based hyperparameter search could also determine whether the ranking persists under settings better suited to each architecture.

The error analysis suggests prioritizing Shirt recognition and confusion among similar garment and

footwear categories. Potential experiments include targeted sampling, moderate class weighting, and changes that preserve more spatial detail. These changes should be assessed using both overall metrics and class-specific precision and recall.

5.3. Conclusion

Under the shared evaluation protocol, the CNN is the most effective configuration for Fashion-MNIST classification. It achieves the highest mean test top-1 accuracy of $91.54 \pm 0.20\%$, the highest mean Macro-F1 of $91.51 \pm 0.19\%$, and the lowest mean test cross-entropy loss of 0.2301 ± 0.0030. Its mean accuracy exceeds the linear classifier by 16.77%.

All evaluated nonlinear models outperform the linear classifier on the overall predictive metrics, supporting the benefit of more expressive representations under the current setup. The LSTM provides a competitive alternative, achieving $90.85 \pm 0.10\%$ accuracy with fewer training epochs than the CNN. However, no model resolves all class-specific errors, and Shirt remains the most challenging category.

The results support the suitability of the CNN configuration for this task, while highlighting persistent confusion between Shirt and visually similar garment classes as a priority for further improvement.

6. AI Usage Disclosure

The group takes full responsibility for all submitted code, numerical results, figures, claims, and text. AI tools supported learning, reviewing and refining member-written code and drafts, building skeleton pages, and language editing. Group members reviewed the adopted suggestions and verified them against the implementation, experimental outputs, and assignment requirements. All reported experimental results were obtained from runs executed by the group, and result figures were produced from those outputs. AI assistance with code or presentation did not replace the execution of experiments or provide fabricated data or results. The following records describe the scope of AI assistance and the corresponding student verification.

6.1. Review and Refinement of the Training and Evaluation Pipeline

- **Tool:** GPT-5.6 Sol.
- **Used by:** Lê Nguyễn Gia Phúc.
- **Development stage:** Pipeline review, refinement, and debugging (Milestone 1).
- **Task:** Reviewing and refining a member-written draft of the FashionMNIST pipeline, covering data preparation, training, evaluation, checkpoint selection, and reproducibility.
- **Prompt summary:**

  Provided an existing member-written pipeline and specified the required workflow, current implementation of EDA, train/validation/test splitting, preprocessing and augmentation, model training pipeline, evaluation metrics, checkpoint selection, and reproducibility. Requested a review of the workflow, explanations of technical issues, and suggestions for improving reproducibility. Also requested appropriate ways to organize and save checkpoints, configurations, experimental outputs.

- **AI contribution:** Reviewed the supplied pipeline and suggested improvements to data preparation, training, evaluation, checkpoint selection, and reproducibility. Recommended saving model configurations, dataset split indices, seed settings, dependency versions, hardware information, checkpoints, metric summaries, and training histories. Suggested linking these artifacts through experiment IDs to support result traceability. Helped modify visualization code for learning curves, confusion matrices, and prediction examples.
- **Student verification:** Compared the suggested changes with the original member-written pipeline and the experimental requirements. Reviewed the data split, preprocessing, training settings, metric calculations, checkpoint-selection rule, and reproducibility settings. Executed the training workflow and inspected the generated metrics and checkpoints.

- **Affected files/sections:** Assignment1/Assignment1.ipynb.
- **Responsible member:** Lê Nguyễn Gia Phúc.

6.2. Student-Proposed Architectures and Technical Verification

- **Tool:** GPT-5.6 Sol
- **Development stage**: Model Implementation (Milestone 1)
- **Used by:** Lê Nguyễn Gia Phúc
- **Task:** Reviewing the student-proposed Linear, MLP, CNN, LSTM, and Transformer architectures for implementation consistency and correctness.
- **Prompt summary:** Proposed the five model architectures and their associated input representations: flattened pixels for Linear and MLP, two-dimensional images for CNN, row sequences for LSTM, and patch tokens for Transformer. Provided model code and requested checks of tensor dimensions, layer connections, dropout placement, and consistency between the model parameters.
- **AI contribution:** Reviewing the configurations, explaining tensor dimensions and layer connections, and suggesting dropout probabilities and placement for regularization. Checked parameter-count calculations and confirmed that the four nonlinear models have approximately comparable trainable parameter counts
- **Student verification:** Compared the programmatically counted trainable parameters across the four nonlinear models to verify that their parameter counts were approximately comparable. Checked that the implemented dropout probabilities and locations matched the selected configurations, and evaluated their combined effect through the dropout ablation study.
- **Affected files/sections:** Assignment1/Assignment1.ipynb.
- **Responsible member:** Lê Nguyễn Gia Phúc.

6.3. Feedback on Results Interpretation and Discussion

- **Tool:** GPT-5.6 Sol
- **Development stage**: Report writing (Milestone 2)
- **Used by:** Lê Nguyễn Gia Phúc.
- **Task:** Reviewing the student's initial interpretations and suggesting improvements to the organization and focus of the comparative discussion.

- **Prompt summary:** Provided the assignment's discussion requirements, the experimental results already obtained, and an initial student-written draft explaining those results. Requested feedback on the draft and suggestions on how to organize and refine the explanations in the report, linking relevant comparisons, observed patterns, and supporting evidence to each requirement. The aim was to keep the discussion focused on the specification and available evidence, while avoiding repetitive descriptions, irrelevant commentary, and unsupported conclusions.
- **AI contribution:** Reviewed the supplied draft and suggested revisions to its structure, wording, and interpretation of the results. Helped connect overall performance, class-specific differences, confusion patterns, training behavior, and computational trade-offs to the assignment requirements. Suggested qualifying explanations related to architecture, regularization, and training where the experiments did not isolate their individual effects.
- **Student verification:** Compared the suggested revisions with the original draft and the assignment's discussion requirements. Checked numerical comparisons against notebook outputs and result tables, and assessed interpretations against the learning curves, confusion matrices, and prediction examples.
- **Affected files/sections:** Report(Implementation Results)
- **Responsible member:** Lê Nguyễn Gia Phúc.

6.4. English Language Editing

- **Tool:** GPT-5.6 Sol.
- **Development stage**: Final Revision
- **Used by:** Hoàng Xuân Bách,Lê Nguyễn Gia Phúc, Nguyễn Việt Hùng
- **Task:** Improving the grammar, clarity, and readability of student-written explanations in the report.
- **Prompt summary:** Provided draft paragraphs written by the student and requested sentence-level language editing: "Fix the grammar and improve the clarity of this paragraph without changing its meaning, technical terminology, or numerical results."
- **AI contribution:** Suggested corrections to grammar, sentence structure, word choice, and transitions. Proposed clearer wording for descriptions of the Linear, MLP, CNN, LSTM, and Transformer results. The requested scope was language editing of the supplied text, with the original interpretations and numerical values preserved.
- **Student verification:** Compared the suggested edits with the original student-written paragraphs.

Checked that model names, technical terminology, numerical values, units, and comparisons remained consistent with the experimental results.

- **Affected files/sections:** Report, Readme.md, assignment1.html
- **Responsible member:** Hoàng Xuân Bách,Lê Nguyễn Gia Phúc, Nguyễn Việt Hùng

6.5. AI-Assisted Github Pages Skeleton

- **Tool:** GPT-5.6 Sol.
- **Development stage**: Planning (Milestone 1)
- **Used by:** Hoàng Xuân Bách, Lê Nguyễn Gia Phúc, Nguyễn Việt Hùng.
- **Task:** Creating the initial Github pages skeleton.
- **Prompt summary:** Requested a standard landing page layout and a basic HTML/CSS skeleton with placeholders.
- **AI contribution:** Assisted with creating the initial landing page skeleton, including basic HTML structure and layout suggestions.
- **Student verification:** Group members reviewed and refactored the generated HTML skeleton to align with the project specifications. Manually populated project-specific content, including team member profiles, course details, and assignment descriptions. Verified the semantic structure, navigation links, and responsive layouts across different screen sizes.
- **Affected files/sections:** index.html, style.css, assignment1.html, assignment2.html, assignment3.html.
- **Responsible members:** Hoàng Xuân Bách, Lê Nguyễn Gia Phúc, Nguyễn Việt Hùng.

6.6. Consistency Verification Between Experimental Outputs and Report

- **Tool:** GPT-5.6 Sol.
- **Development stage**: Final Revision (Milestone 2)
- **Used by:** Lê Nguyễn Gia Phúc.
- **Task:** Supporting the final consistency check between experimental outputs and the report's implementation results.
- **Prompt summary:** Provided the final experimental outputs and the corresponding report content. Requested assistance with checking numerical consistency and identifying outdated values,

mismatched labels, or explanations that were not supported by the results.

- **AI contribution:** Assisted with reviewing metric values, mean and sample standard deviation, parameter counts, computational costs, and Baseline/NoDropout comparisons. Suggested checking that tables, figures, captions, and conclusions referred to the same experimental configurations and aggregation conventions.
- **Student verification:** Compared the report's numerical statements, tables, figures, and captions with the final notebook outputs. Checked seed settings, variant labels, units, and aggregation conventions. Corrected outdated or inconsistent content and reviewed the conclusions against the experimental evidence.
- **Affected files/sections:** Report(Implementation Results)
- **Responsible member:** Lê Nguyễn Gia Phúc.

6.7. Consistency Verification Between Experimental Outputs and GitHub Pages

- **Tool:** GPT-5.6 Sol.
- **Development stage**: Final Revision (Milestone 2)
- **Used by:** Hoàng Xuân Bách.
- **Task:** Supporting the final consistency check between experimental outputs and the results presented on GitHub Pages.
- **Prompt summary:** Provided the final experimental outputs and the corresponding GitHub Pages content. Requested assistance with checking whether the published numerical summaries, tables, figures, and explanations accurately represented the completed experiments.
- **AI contribution:** Assisted with reviewing the consistency of displayed metrics, model names, variant labels, units, and figure descriptions. Suggested checking that published figures and summaries used the final results, with single-seed examples distinguished from results aggregated across three seeds.
- **Student verification:** Compared the Assignment 1 results displayed on GitHub Pages with the final notebook outputs. Checked numerical summaries, table entries, figure versions, captions, and model labels. Corrected outdated content and inspected the rendered pages to confirm that the updated results and figures were displayed correctly.
- **Affected files/sections:** assignment1.html
- **Responsible member:** Hoàng Xuân Bách.

6.8. LaTeX Formatting Assistance

- **Tool:** GPT-5.6 Sol.
- **Used by:** Lê Nguyễn Gia Phúc, Hoàng Xuân Bách, Nguyễn Việt Hùng
- **Development stage:** Report formatting and final revision (Milestone 2).
- **Task:** Improving the LaTeX presentation of student-provided report content.
- **Prompt summary:** Provided existing text, tables, and equations and requested formatting assistance. Requests included converting the experimental design into a comparison table, adjusting column widths and text alignment, splitting long tables, and formatting mathematical expressions without changing their intended meaning or numerical values.
- **AI contribution:** Suggested LaTeX environments and formatting code for tables, equations, and structured lists. Helped arrange the main comparison and dropout ablation side by side, merge shared experimental settings, and improve the readability of lengthy tables. Provided revised LaTeX snippets based on the supplied content.
- **Student verification:** Compiled the revised LaTeX and inspected the rendered report. Compared the formatted content with the original text and experimental outputs. Checked table entries, mathematical symbols, captions, labels, and cross-references, and adjusted spacing and page breaks where necessary.
- **Affected files/sections:** Report LaTeX source, particularly the experimental design, result tables, architecture equations, and AI-use disclosure sections.
- **Responsible member:** Lê Nguyễn Gia Phúc, Hoàng Xuân Bách, Nguyễn Việt Hùng